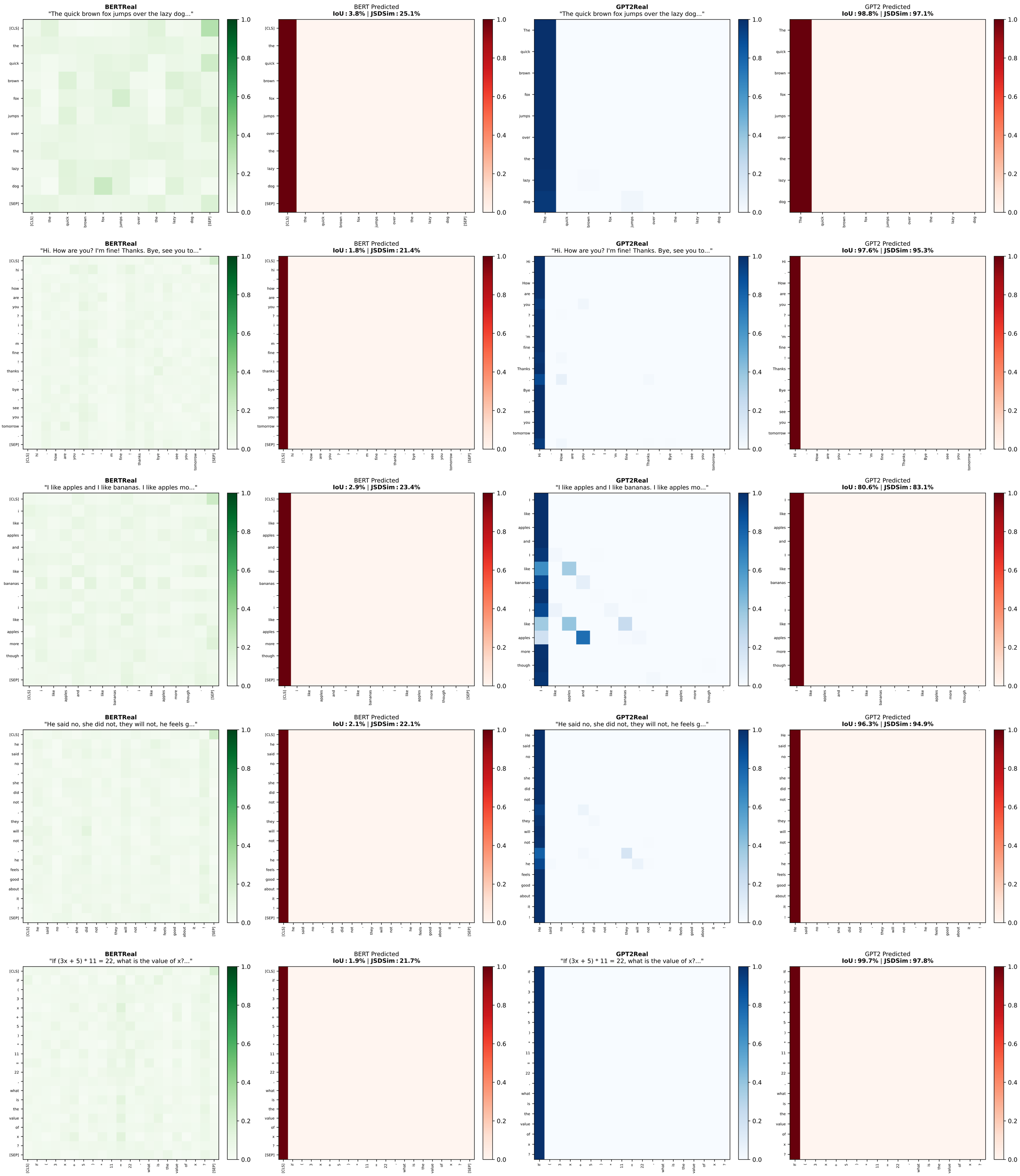
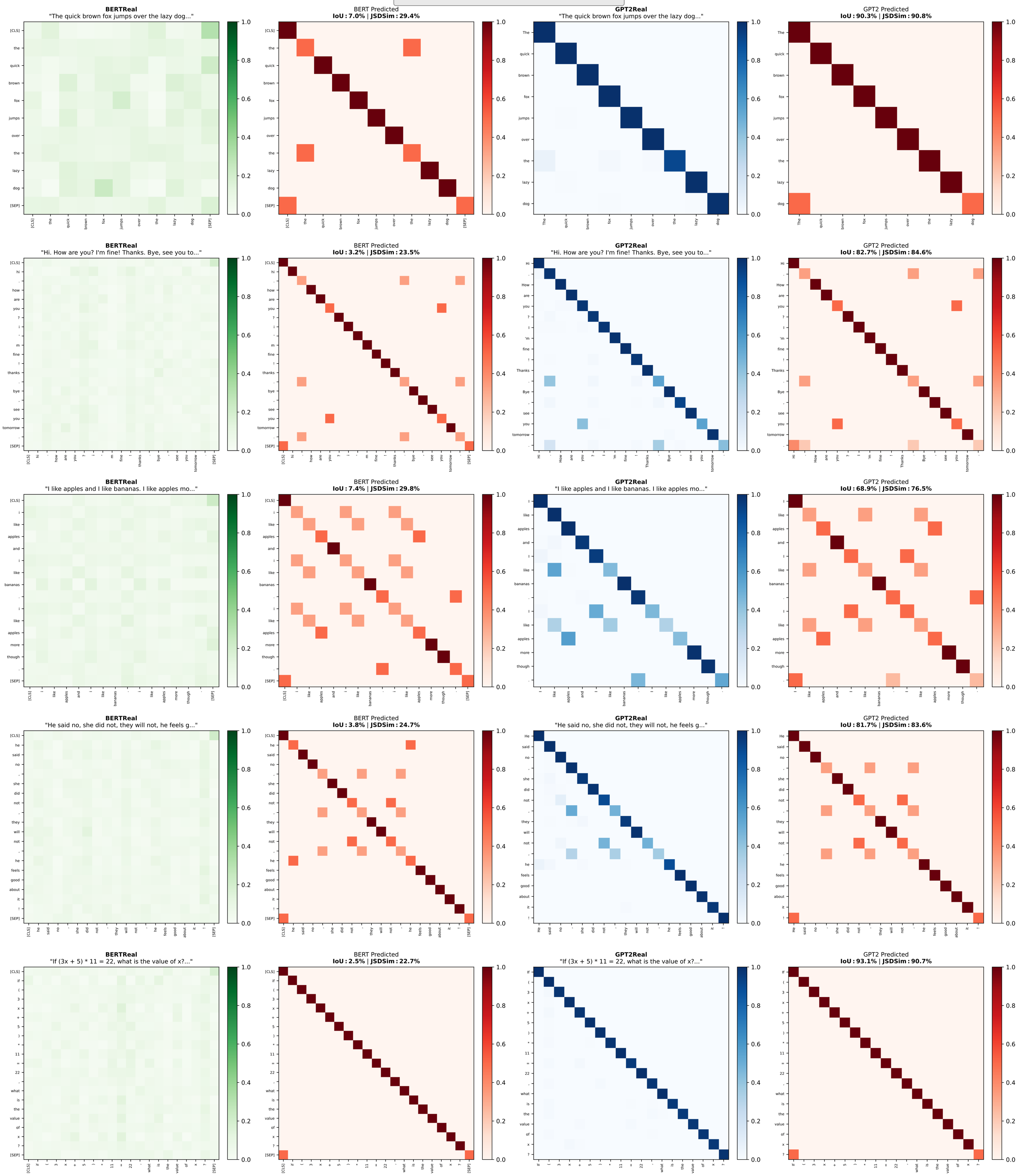
Pattern: leading_contextual_emphasis

```
def leading_contextual_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase) -> str:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    # Assume the onset of context is emphasized
    for i in range(1, len_seq-1):
        out[i, 0] = 1  # Emphasize connection from CLS (first token)
        out[0, 0] = 1  # Self-attention
        out[-1, 0] = 1  # Emphasize when reaching the output token
    return "Leading Contextual Emphasis", out
```



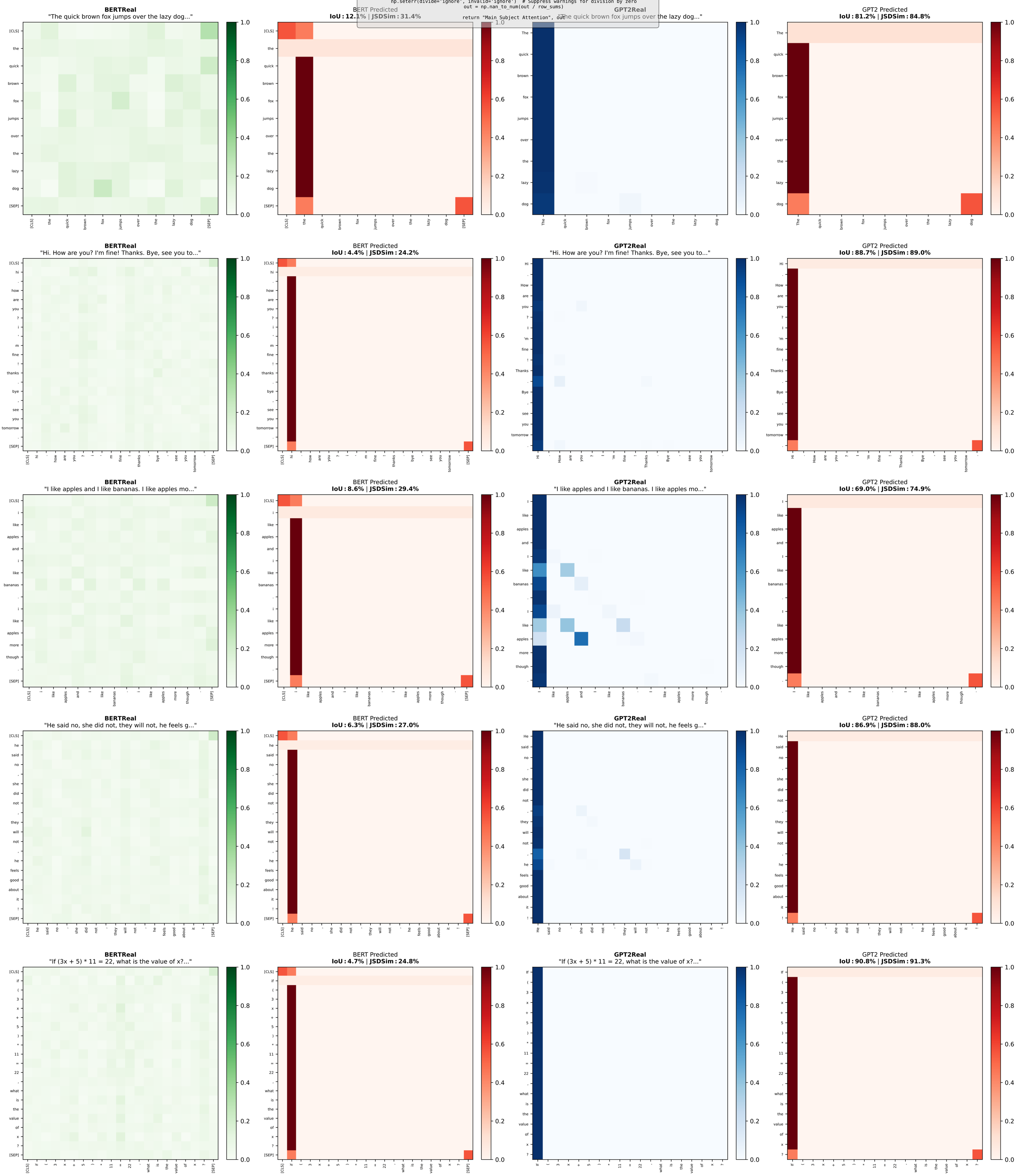
Pattern: lexical_diversity_focus

```
def lexical_diversity_focus(sentence: str, tokenizer: PreTrainedTokenizerBase):
                toks = tokenizer([sentence], return_tensors="pt")
                len_seq = len(toks.input_ids[0])
                out = np.zeros((len_seq, len_seq))

        # Tokenize the sentence with tokenizer and identify unique tokens
           tokens = tokenizer.convert_ids_to_tokens(toks.input_ids[0])
                unique_tokens = list(set(tokens))

            # Map each unique token its occurrences in the sentence
               token_map = {token: [] for token in unique_tokens}
                    for i, token in enumerate(tokens):
                            if token in token_map:
                            token_map[token].append(i)

          # Assign high attention to unique tokens and medium to others
                    for token, indices in token_map.items():
                                    for idx in indices:
                            if len(indices) == 1:  # Unique token
                                            out[idx, idx] = 1
                                    else:  # Non-unique token
                                        attention_value = 0.5
                                        for i_idx in indices:
                                    out[i_idx, idx] = attention_value

                    # Assign attention to [CLS] and [EOS] tokens
                                    out[0, 0] = 1
                                    out[-1, 0] = 1

                            # Normalize the matrix
                    out = out / out.sum(axis=1, keepdims=True)
            return "Lexical Diversity Focus and Attention Pattern", out
```



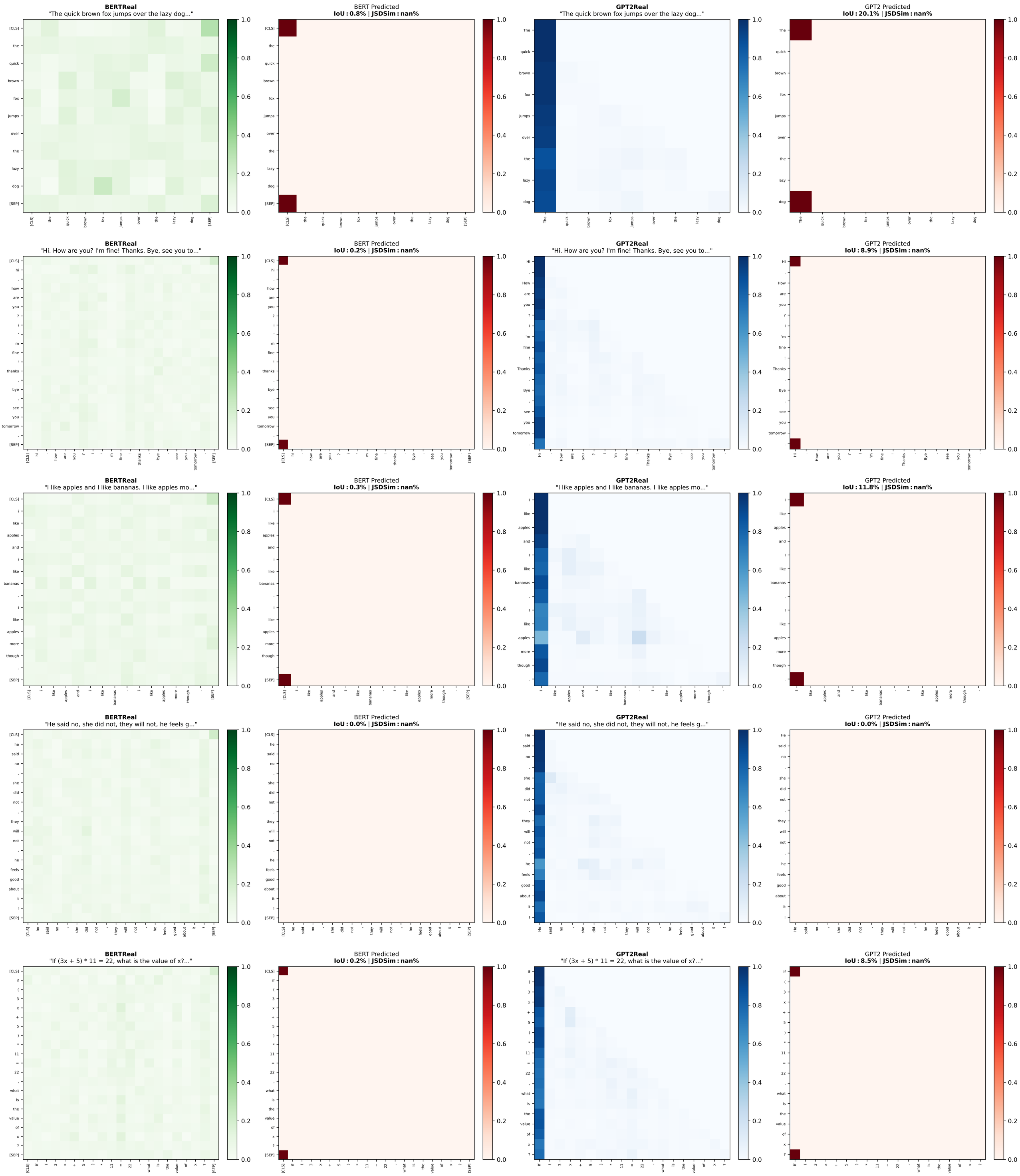
Pattern: main_subject_attention

```
def main_subject_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                    # Tokenize sentence
                            toks = tokenizer([sentence], return_tensors="pt")
                                   len_seq = len(toks.input_ids[0])
                                  out = np.zeros((len_seq, len_seq))

                        # Find the first token in the sentence (usually the main subject)
            # Assuming the first content word after specials and determiners is the main subject
                                 token_ids = toks.input_ids[0].tolist()
                 special_token_id = tokenizer.cls_token_id if tokenizer.cls_token_id else 0
                                      main_subject_index = 0

                                       # Skip special tokens
                                for i, token_id in enumerate(token_ids):
                                      if token_id != special_token_id:
                                            main_subject_index = i
                                                    break

            # Create a mapping for attention where the main subject token attends to most tokens
                     # Normalize attention so that the main subject has the most focus
                                     for i in range(len_seq):
                                      if i == main_subject_index:
                                       out[main_subject_index, :] = 1
                                                  else:
                                     out[i, main_subject_index] = 0.8

                                  # Set CLS and EOS token attention
                                           out[0, 0] = 1
                                          out[-1, -1] = 1

                                   # Normalize the output matrix by rows
                                 row_sums = out.sum(axis=1, keepdims=True)
            np.seterr(divide='ignore', invalid='ignore')  # Suppress warnings for division by zero
                                  out = np.nan_to_num(out / row_sums)

                        return "Main Subject Attention", out
```



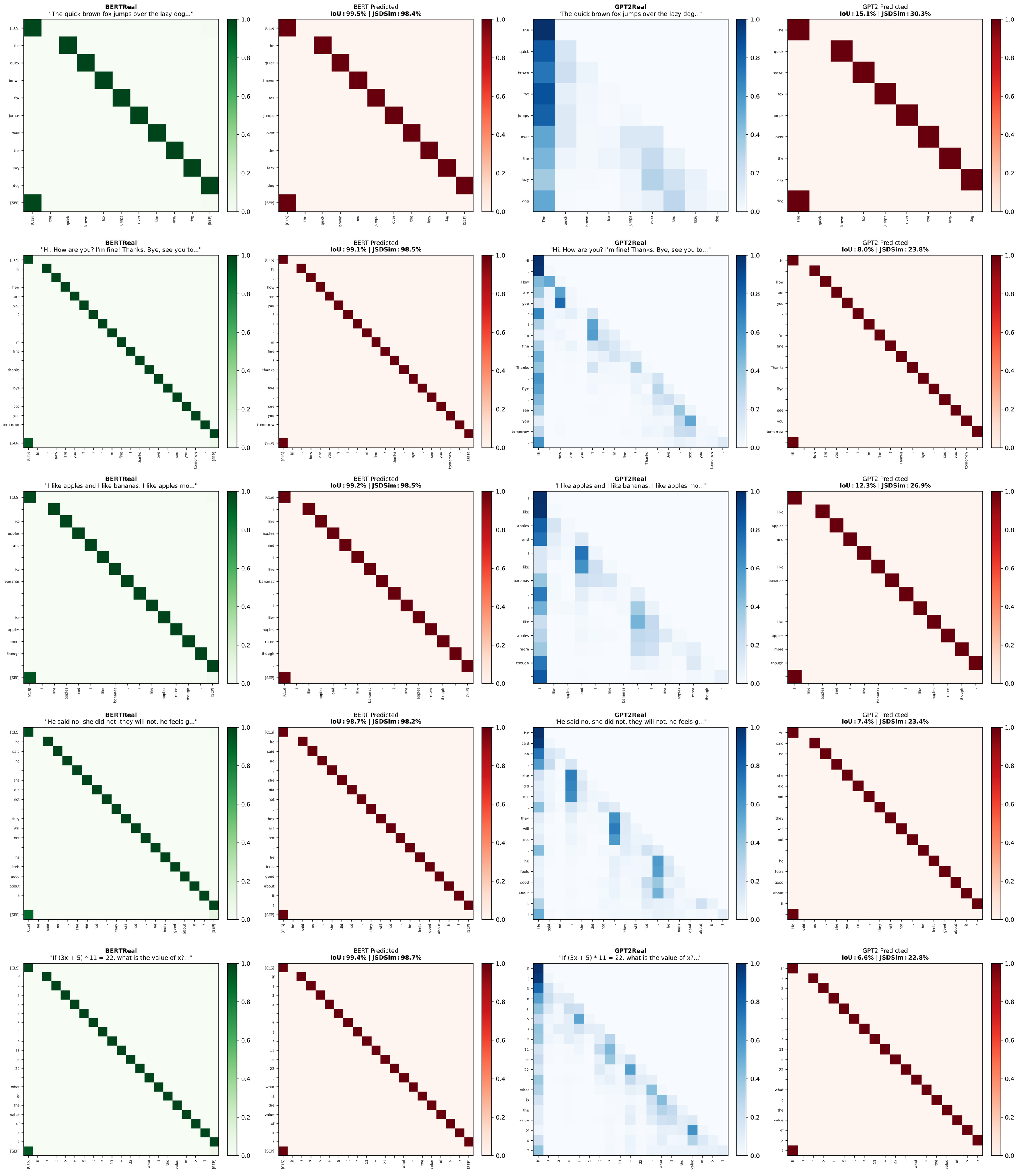
Pattern: negation_attention

```
def negation_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    input_ids = toks.input_ids[0].tolist()
    len_seq = len(input_ids)
    out = np.zeros((len_seq, len_seq))
    doc = nlp(sentence)
    for token in doc:
    if token.dep_ == "neg":
    token_idx = sentence.split().index(token.text)
    out[token_idx, :] = 1
    out[0, 0] = 1
    out[-1, 0] = 1
    return "Negation Token Pattern", out
```



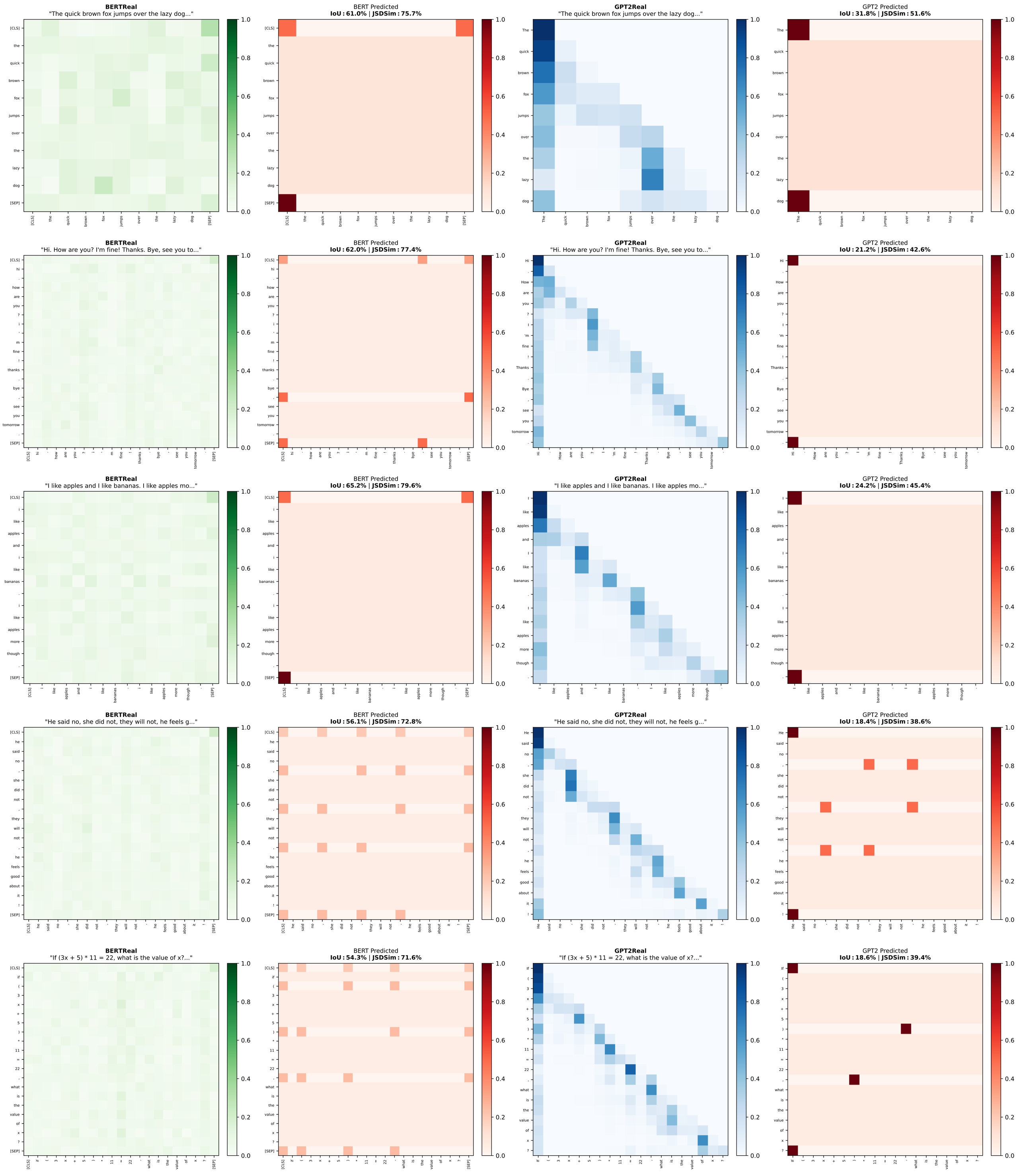
Pattern: next_attention

```
def next_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    for i in range(1, len_seq-1):
        out[i, i+1] = 1
        out[0,0] = 1
        out[-1,0] = 1
    return "Next Token Pattern", out
```



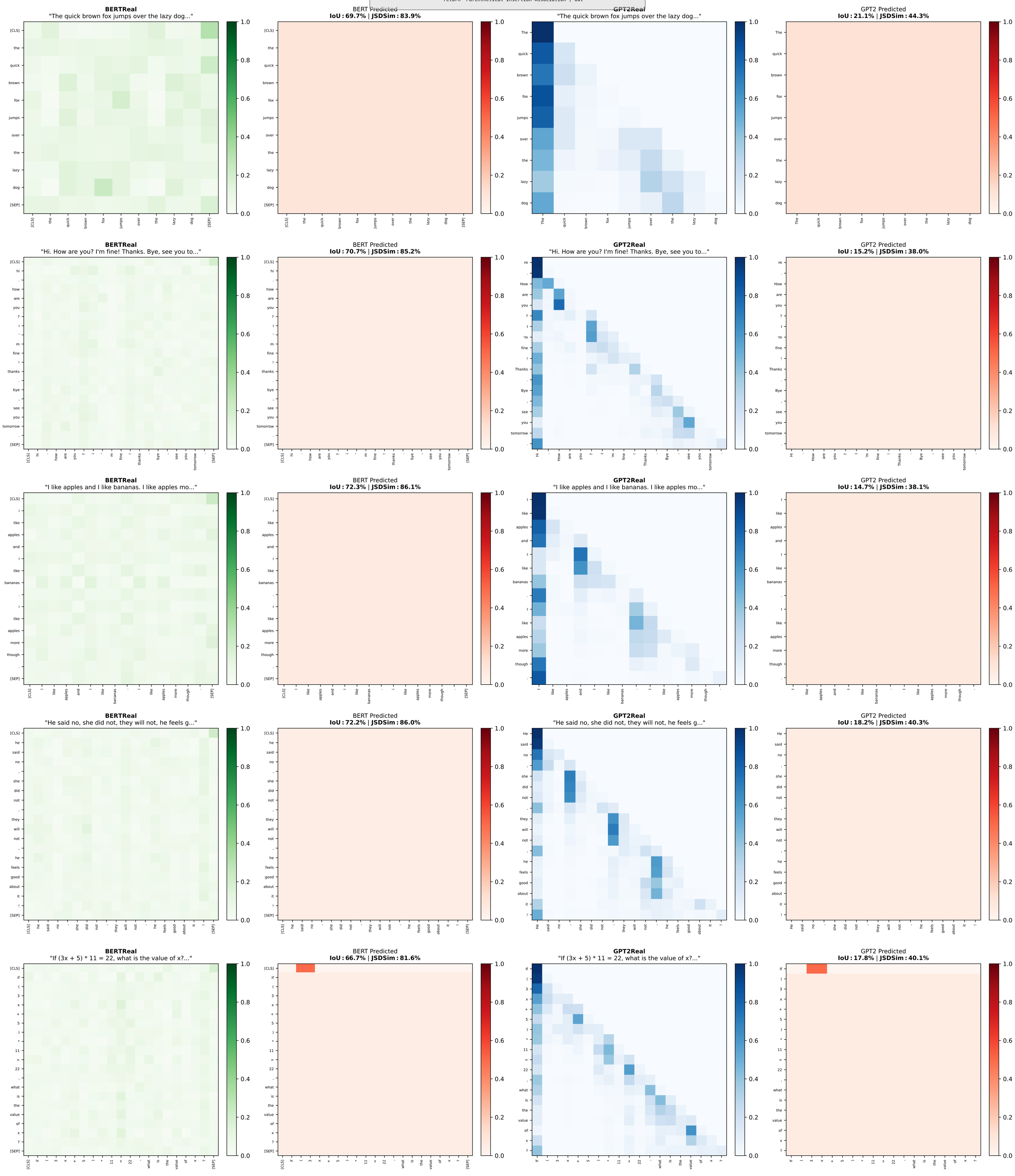
Pattern: parenthetical_attention

```
def parenthetical_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors='pt')
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    words = tokenizer.convert_ids_to_tokens(toks.input_ids[0])

    paren_indices = []

    # find indices for parenthetical commas or phrases
    for i, word in enumerate(words):
        if word in {',', '(', ')', '[SEP]', '[CLS]'} or word.endswith(','):  # use endswith to cover subword tokens ending in comma
            paren_indices.append(i)

    # Connect parenthetical commas with each other
    for i in paren_indices:
        for j in paren_indices:
            if i != j:
                out[i][j] = 1

    # Add attention from the start token and stop token to the rest
    out[0, 0] = 1
    out[-1, 0] = 1

    # Normalize
    out += 1e-4
    out = out / out.sum(axis=1, keepdims=True)

    return 'Parenthetical Phrase Attention', out
```



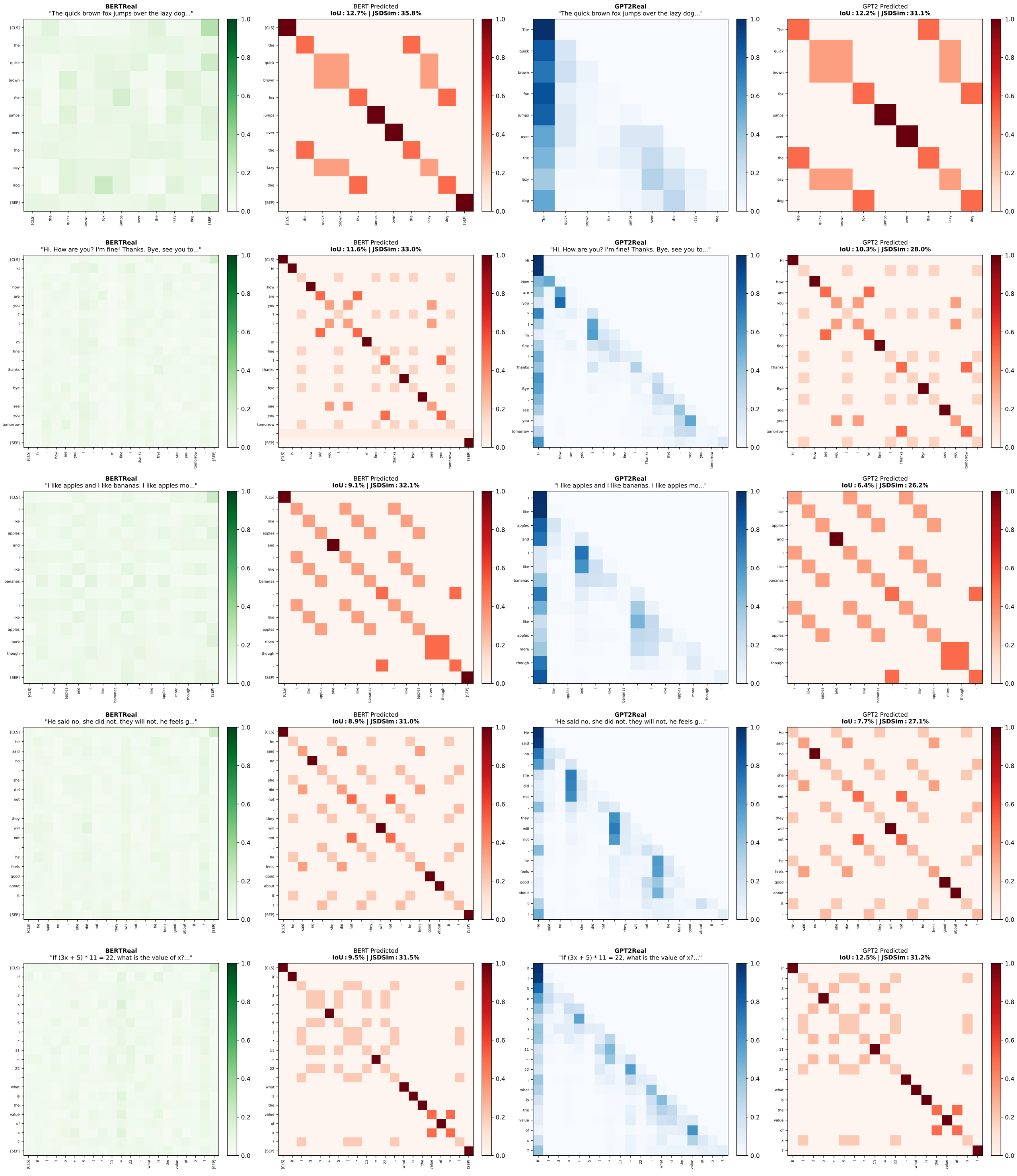
Pattern: parenthetical_insertion

```
def parenthetical_insertion(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Placeholder tokens for parenthetical phrases often include commas and parenthesis
    words = sentence.split()
    start = None
    pairings = []

    for i, word in enumerate(words):
        # If a parenthetical-like context starts
        if word.startswith('(') or word.startswith(','):
            if not start:
                start = i + 1
                continue

        # If a parenthetical-like context ends
        if word.endswith(')') or word.endswith(','):
            if start is not None:
                pairings.append((start, i + 1))
                start = None

    # Creating the matrix based on detected parenthetical associations
    for start, end in pairings:
        for i in range(start, end):
            out[0, i] = 1  # Attend mainly between separators and the first token

    # Add a small value to non-zero elements to ensure we don't end with sparse matrices
    out = np.clip(out, 1e-4, 1.0)
    out = out / out.sum(axis=1, keepdims=True) if out.sum(axis=1, keepdims=True).all() else out

    return "Parenthetical Insertion Association", out
```



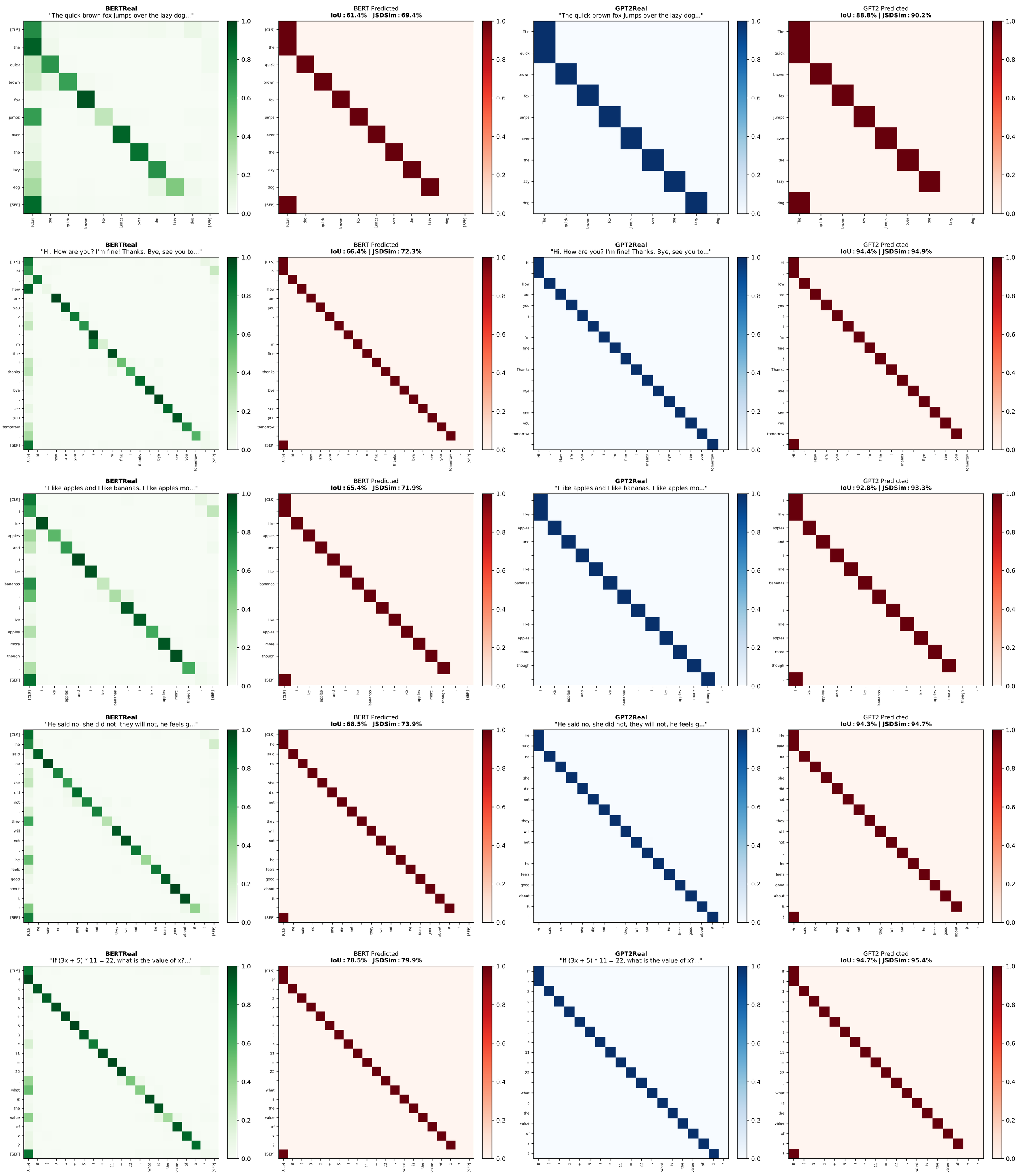
Pattern: pos_alignment

```
def pos_alignment(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
toks = tokenizer(sentence, return_tensors="pt", return_offsets_mapping=True, add_special_tokens=True)
input_ids = toks.input_ids[0].tolist()
word_ids = toks.word_ids(0)
len_seq = len(input_ids)
out = np.zeros((len_seq, len_seq))
doc = nlp(sentence)
pos_tags = [token.pos_ for token in doc]
pos_to_token_indices = {}
for token_idx, word_idx in enumerate(word_ids):
if word_idx is None or word_idx >= len(pos_tags):
continue
pos = pos_tags[word_idx]
pos_to_token_indices.setdefault(pos, []).append(token_idx)
for token_indices in pos_to_token_indices.values():
if len(token_indices) > 1:
for i in token_indices:
for j in token_indices:
out[i, j] = 1
else:
i = token_indices[0]
out[i, i] = 1
out[0, 0] = 1
out[-1, -1] = 1
out += 1e-4
out = out / out.sum(axis=1, keepdims=True)
return "Part of Speech Pattern", out
```



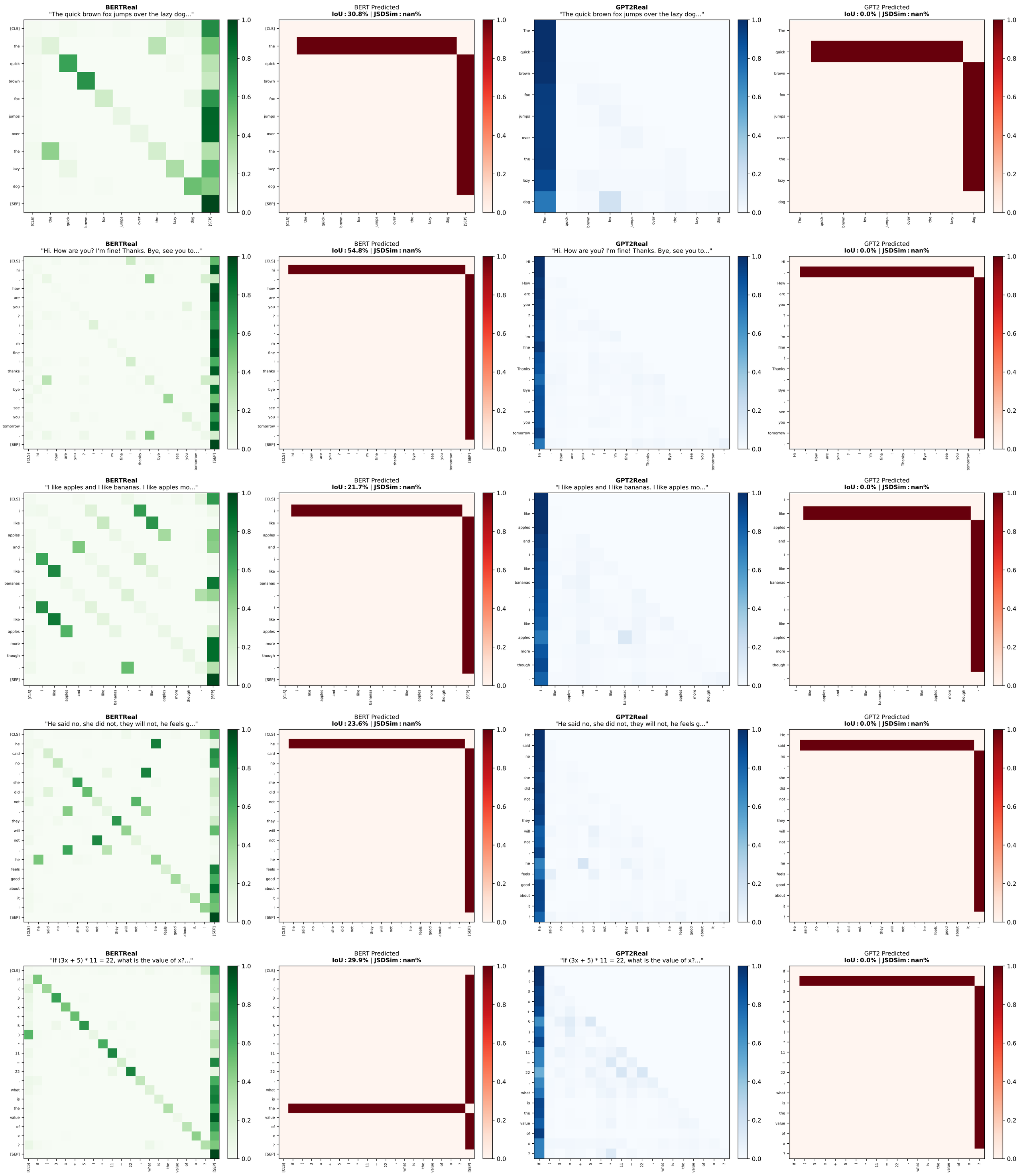
Pattern: previous_attention

```
def previous_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    for i in range(1, len_seq-1):
        out[i, i-1] = 1
        out[0,0] = 1
        out[-1,0] = 1
    return "Previous Token Pattern", out
```



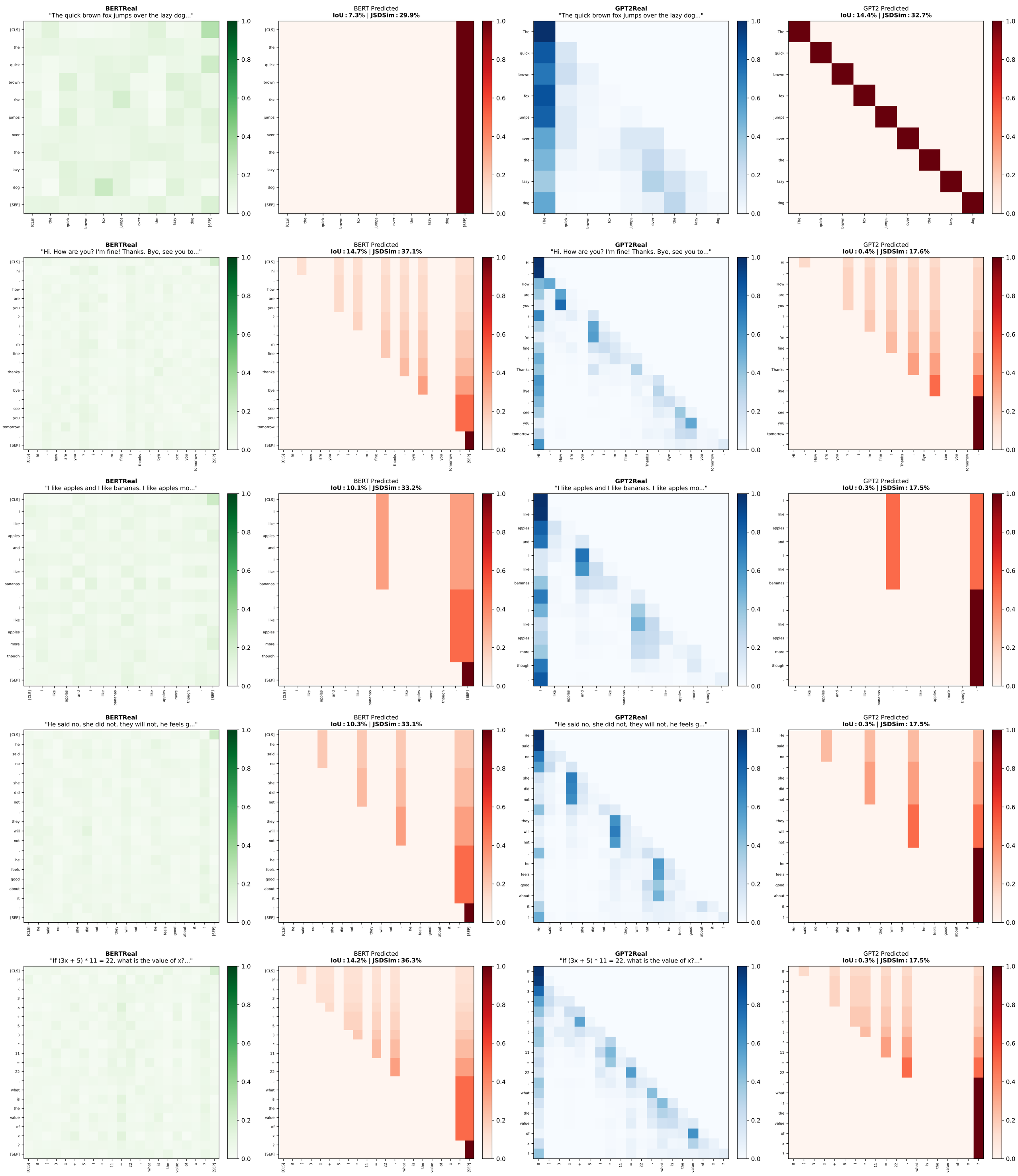
Pattern: pronoun_reference

```
def pronoun_reference(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
token_nouns = {'they', 'she', 'he', 'it', 'that', 'this', 'these', 'those', 'we', 'us', 'you', 'them', 'me', 'i', 'him', 'her', 'one', 'someone', 'everyone', 'no one', 'anyone', 'thing'}

    # Find main token to focus attention
    focus_token_index = 1  # Default focus on the first meaningful token (after CLS)
    for i, token_text in enumerate(tokenizer.convert_ids_to_tokens(toks.input_ids[0])):
        if i == 0:
            continue
        if any(pronoun.lower().lstrip().startswith(token_text.strip().lower()) for pronoun in token_nouns):
            focus_token_index = i
            break

    # The main token (typically a pronoun at start) spreads attention across the sentence.
    out[focus_token_index, 1:-1] = 1.0  # Spread attention over all tokens excluding CLS and possibly EOS

    # Ensure any sentence's attention is balanced across.
    for row in range(1, len_seq - 1):  # Exclude CLS, EOS
        if out[row].sum() == 0:
            out[row, -1] = 1.0  # Assign any remaining empty row attention to EOS

    return "Sentence-level Pronoun Reference Pattern", out
```



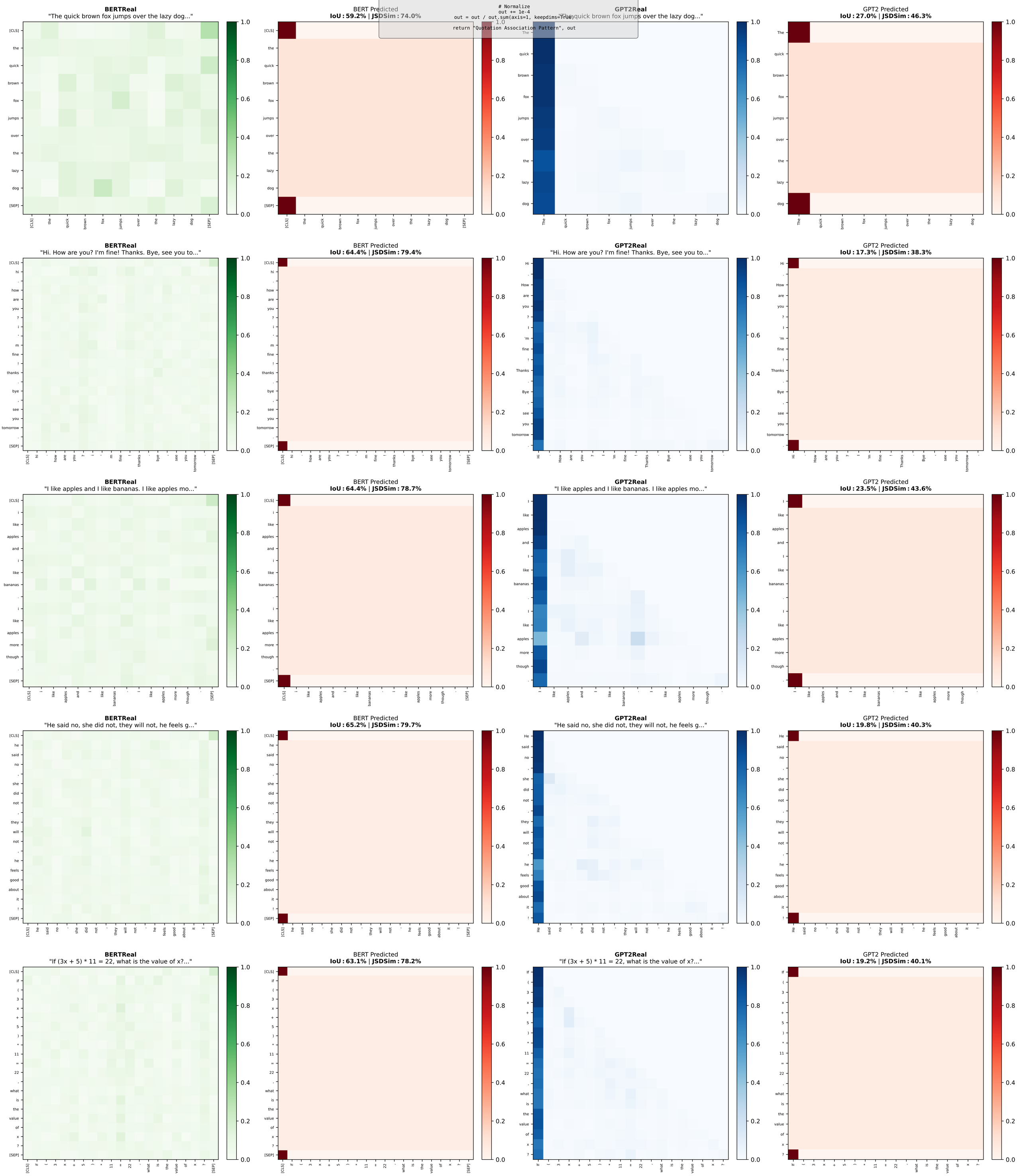
Pattern: punctuation_attention

```
def punctuation_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                         len_seq = len(toks.input_ids[0])
                                        out = np.zeros((len_seq, len_seq))
                          words = tokenizer.convert_ids_to_tokens(toks.input_ids[0])
                         punctuation_set = set('!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~')
    punctuation_indices = [i for i, tok in enumerate(words) if any(p in tok for p in punctuation_set)]
                                         for i in range(len_seq):
                             future_punct = [j for j in punctuation_indices if j > i]
                                                if future_punct:
                                               for j in future_punct:
                                                    out[i, j] = 1.0
                                               out[i] /= out[i].sum()
                                                    else:
                                                  out[i, i] = 1.0
                                                 out += 1e-4
                                  out = out / out.sum(axis=1, keepdims=True)
                                      return "Punctuation Pattern", out
```



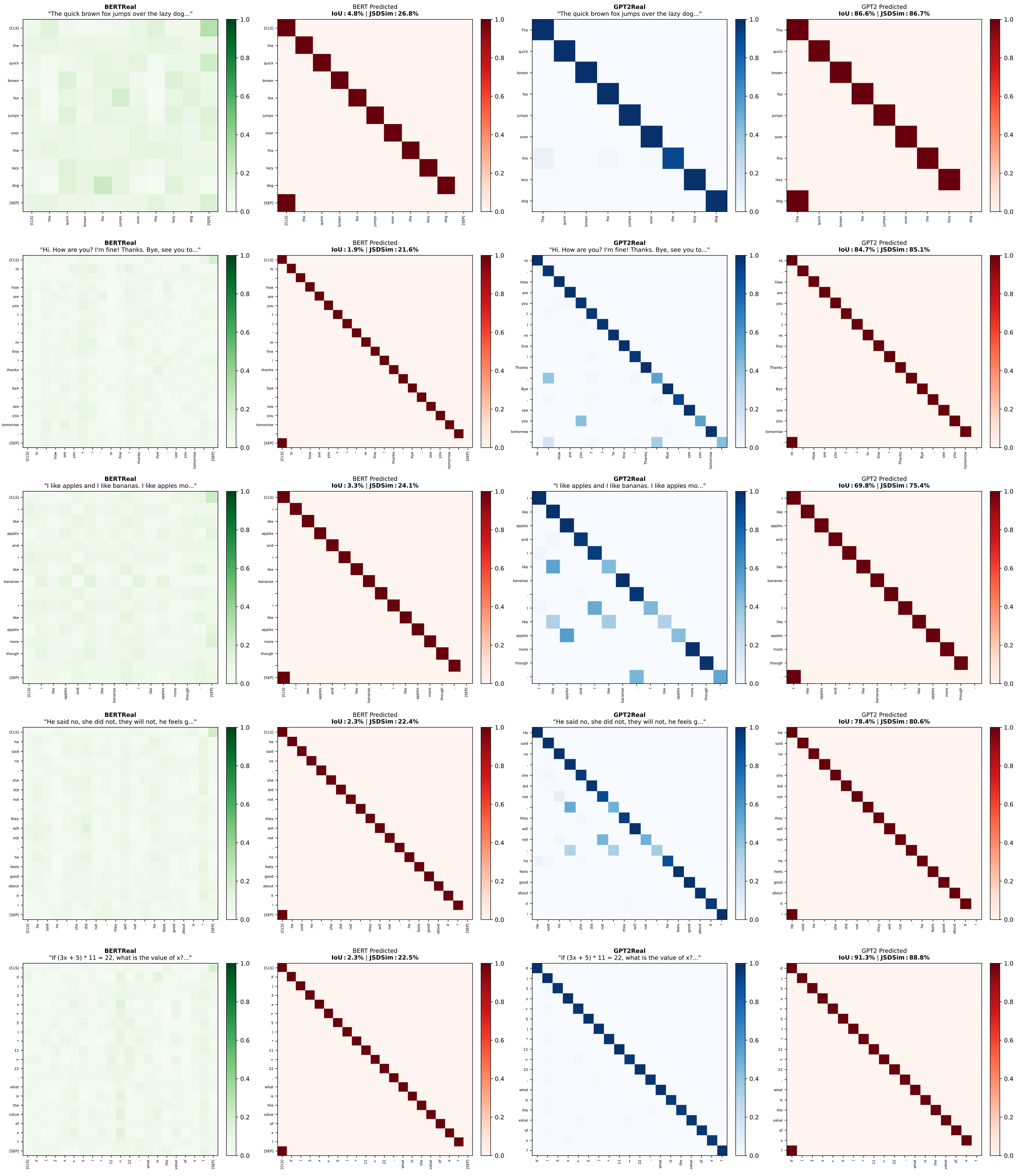
Pattern: quotation_association

```
def quotation_association(sentence: str, tokenizer) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Tokenize sentence and detect quotation marks
    tokens = sentence.split()
quote_indices = [i for i, token in enumerate(tokens) if "'" in token or '"' in token]
    quote_pairs = []

    # Pair the indices assuming quote-indexed pairs
    for i in range(0, len(quote_indices), 2):
        if i+1 < len(quote_indices):
quote_pairs.append((quote_indices[i], quote_indices[i+1]))

    # Mapping tokens according to tokenizer
    word_to_tokens = []
    for word in tokens:
        current_token_id = len(word_to_tokens)
        for token in tokenizer.tokenize(word):
            word_to_tokens.append(current_token_id)

    # Apply quote attention
    for (q_start, q_end) in quote_pairs:
        tok_start = word_to_tokens[q_start]
        tok_end = word_to_tokens[q_end]
out[tok_start + 1, tok_start + 1 : tok_end + 2] = 1
out[tok_end + 1, tok_start + 1 : tok_end + 2] = 1

    # Mark [CLS] and [SEP]
    out[0, 0] = 1
    out[-1, 0] = 1

    # Normalize
    out += 1e-4
out = out / out.sum(axis=1, keepdims=True)
return "Quotation Association Pattern", out
```



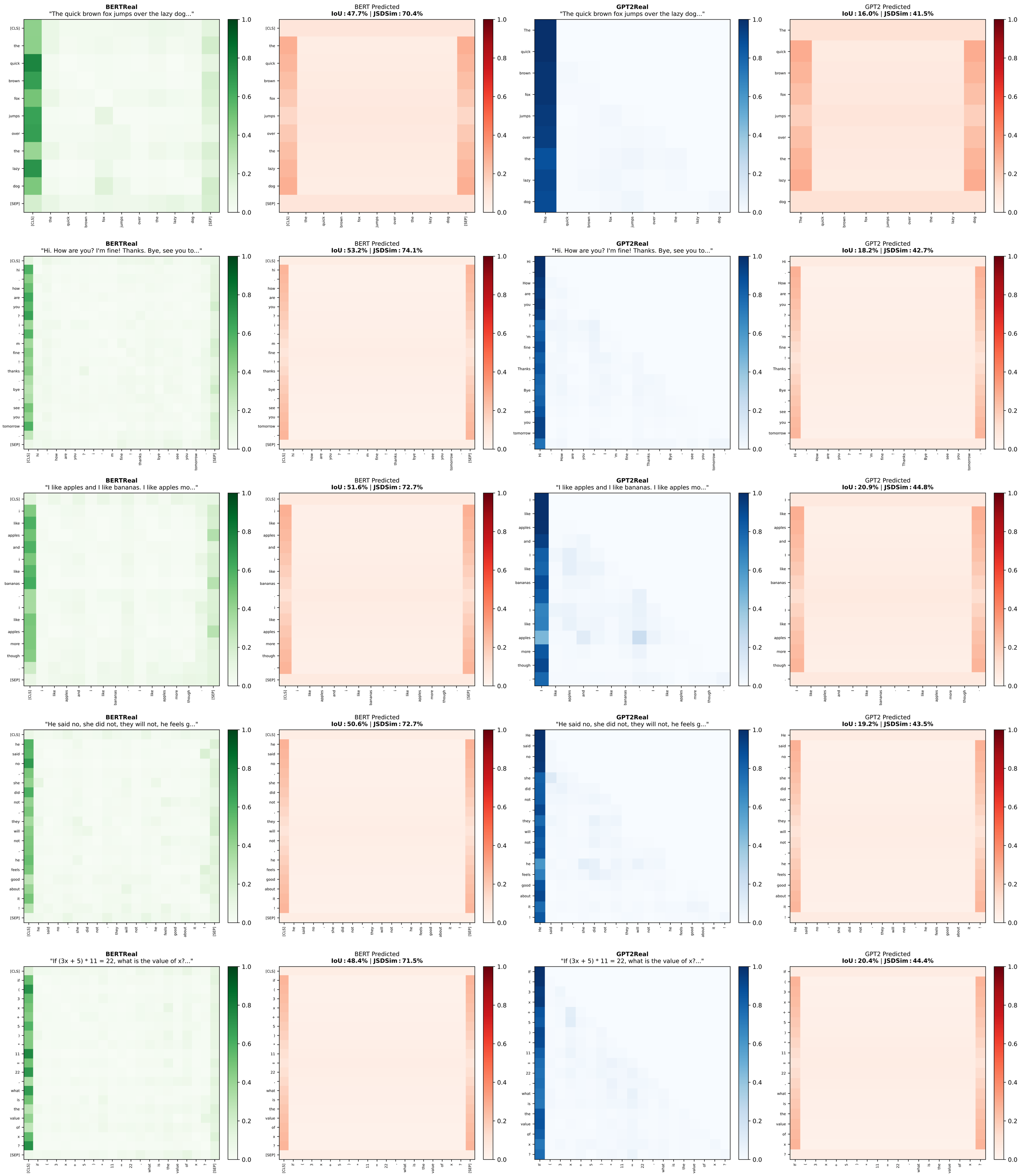
Pattern: rare_word_dominance

```
def rare_word_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

# A placeholder mechanism to weight tokens based on a pseudo-frequency estimate
# Here, the assumption is that rarer words within their context get higher attention.
# The toy example uses simple rules since real data is inaccessible.
    for token_idx in range(1, len_seq - 1):
        # Let's assume the weights are inversely proportional to the index
        # In reality, you'd access frequency data or an equivalent method
        out[token_idx, token_idx] = 1.0 / (token_idx + 1)

    out[0, 0] = 1  # Self-attention to CLS
    out[-1, 0] = 1  # EOS pattern recognization

# Normalize rows of the matrix as attention heads do in practice
    out = out / out.sum(axis=1, keepdims=True)
    return "Rare Word Dominance", out
```



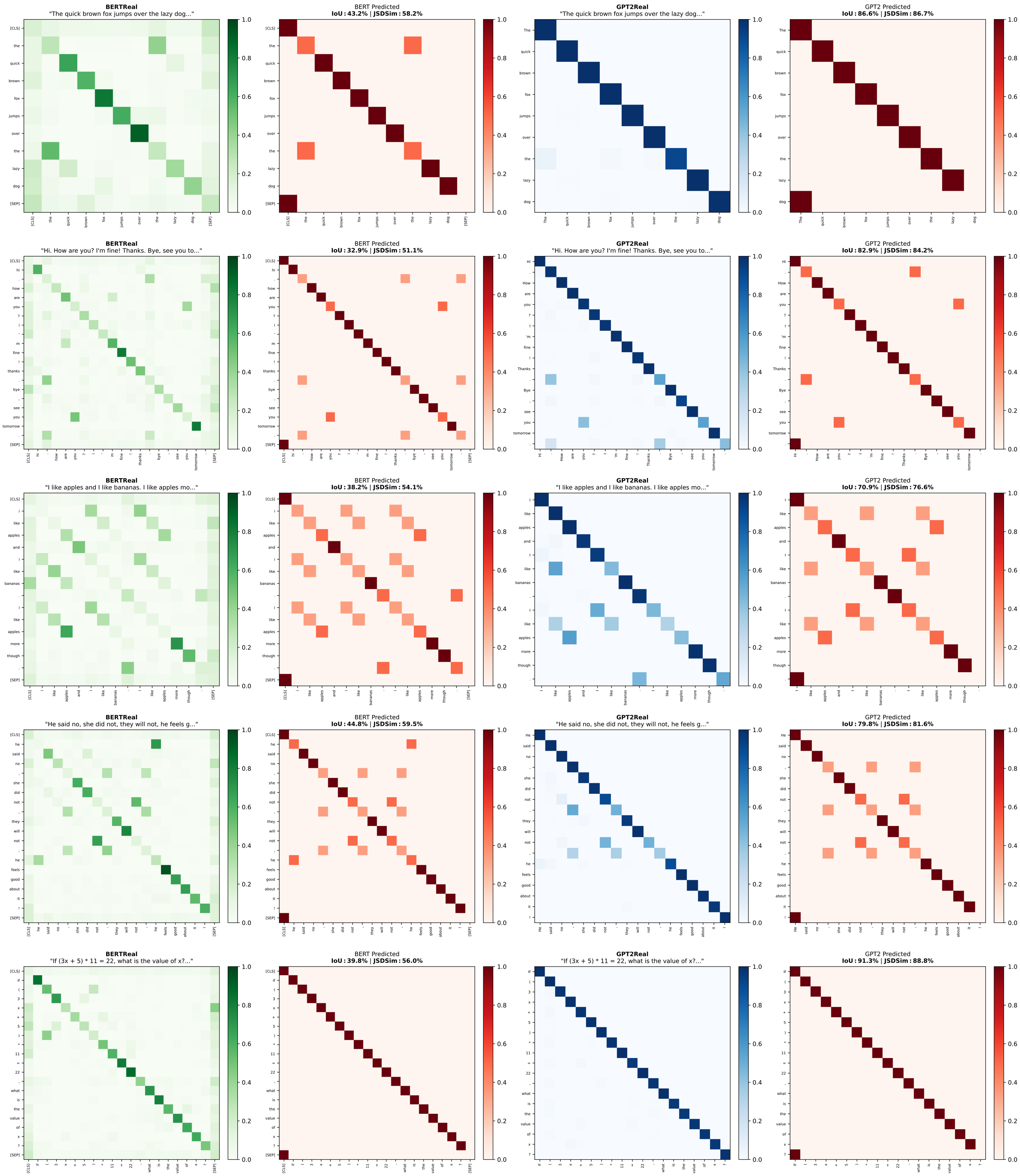
Pattern: relative_position_attention

```
def relative_position_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))

            # Assign relative position importance to special tokens [CLS] and [SEP]
                        out[0, :] = 1   # [CLS] self-attention
                        out[:, 0] = 1   # [CLS] attends to all

                        out[-1, :] = 1  # [SEP] self-attention
                        out[:, -1] = 1  # [SEP] attends to all

                # Calculate relative distance decay, favoring 'central' tokens
                                center = len_seq // 2
                            for i in range(1, len_seq-1):
                           dist_from_center = abs(center - i)
                     decayed_importance = 1 / (1 + dist_from_center)
                             out[i, :] += decayed_importance

                # Normalize out matrix by row to simulate attention distribution
                         out = out / out.sum(axis=1, keepdims=True)
                          return "Relative Position Attention", out
```



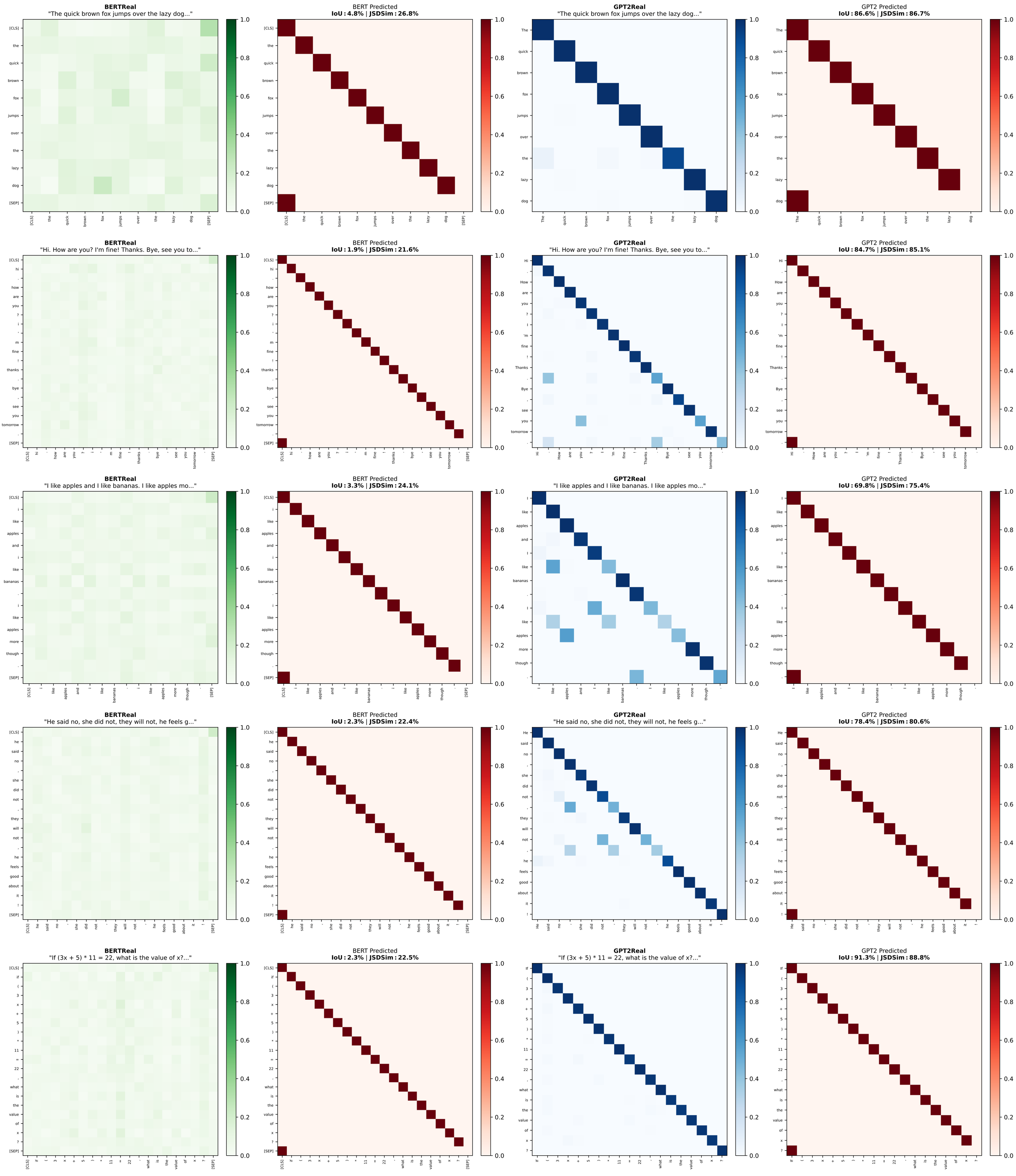
Pattern: repeated_attention

```
def repeated_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    input_ids = toks.input_ids[0].tolist()
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    for i in range(1, len_seq-1):
        token_id = input_ids[i]
        for j in range(1, len_seq-1):
            if input_ids[j] == token_id:
                out[i, j] = 1
    out[0,0] = 1
    out[-1,0] = 1
    out = out / out.sum(axis=1, keepdims=True)
    return "Repitition Pattern", out
```



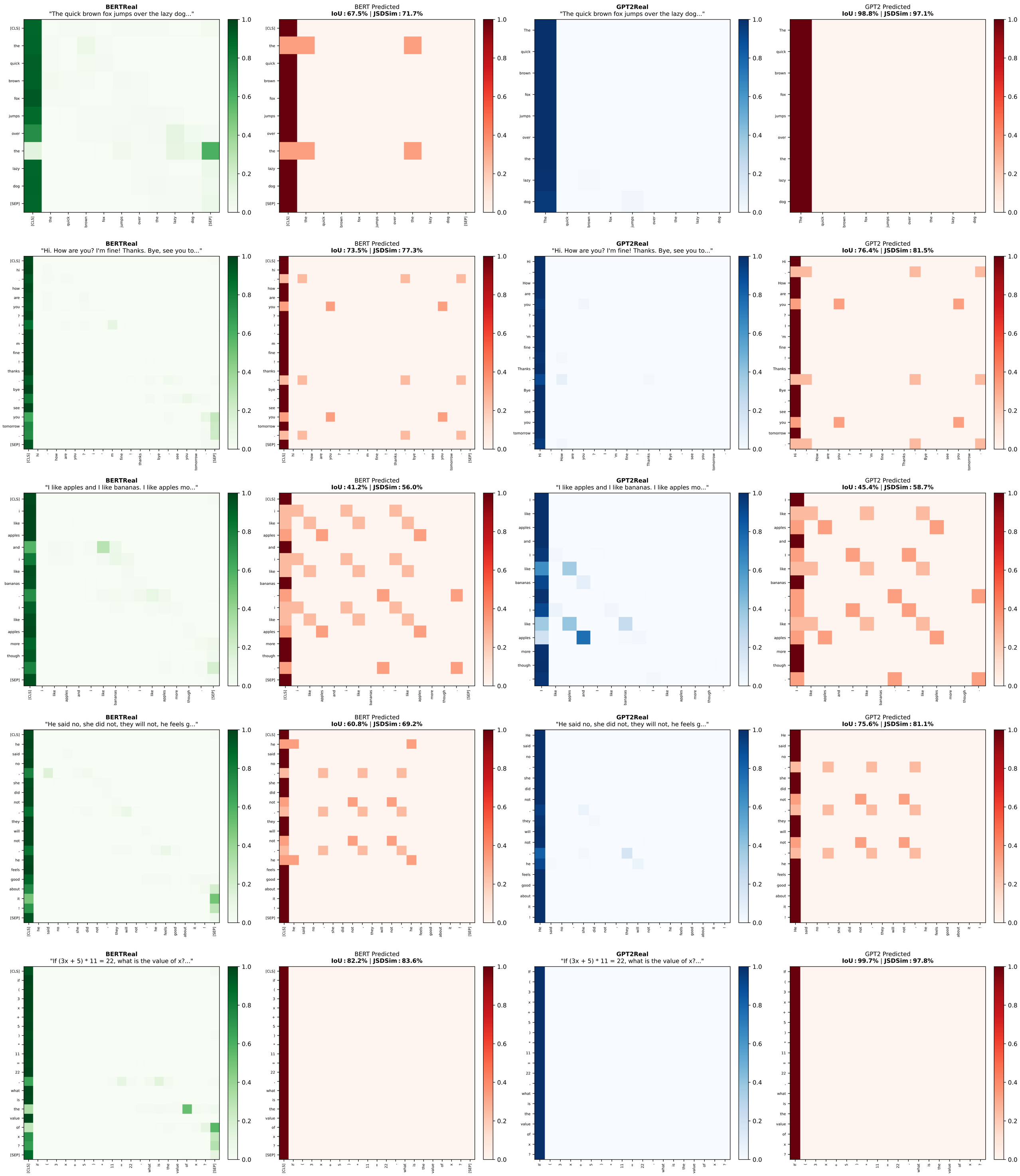
Pattern: same_attention

```
def same_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    for i in range(1, len_seq-1):
        out[i, i] = 1
    out[0,0] = 1
    out[-1,0] = 1
    return "Same Token Pattern", out
```



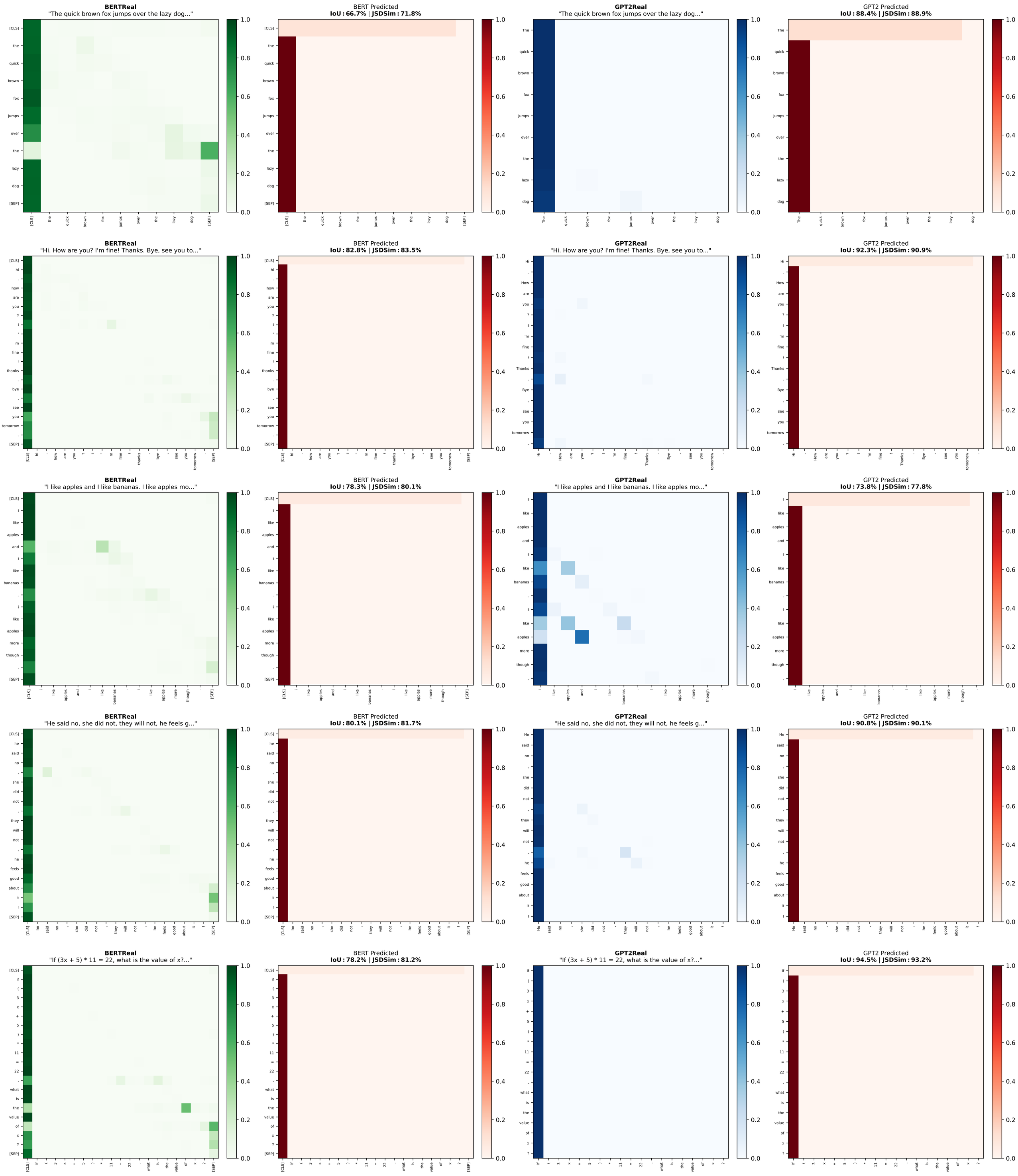
Pattern: semantic_grouping

```
def semantic_grouping(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    tokens = tokenizer.convert_ids_to_tokens(toks.input_ids[0])
    attention_dict = {}

    # Hypothesis is that certain content words like nouns or subjects are attracting initial tokens
    for idx, token in enumerate(tokens):
        out[idx, 0] = 1
        if token not in attention_dict:
            attention_dict[token] = [idx]
        else:
            attention_dict[token].append(idx)

    # Map first non-punctuation token to each token in its respective semantic group
    for token, indices in attention_dict.items():
        if len(indices) > 1:
            for idx in indices:
                for j in indices:
                    out[idx, j] = 1
                    out[j, idx] = 1

    # CLS and SEP tokens receive their own distinct attention, marked at the [0][0] and last[0] index.
    out[0, 0] = 1
    out[-1, 0] = 1
    # Normalize the attention distribution by row to have a uniform attention weight sum
    out = out / out.sum(axis=1, keepdims=True)
    return "Semantic Grouping Pattern", out
```



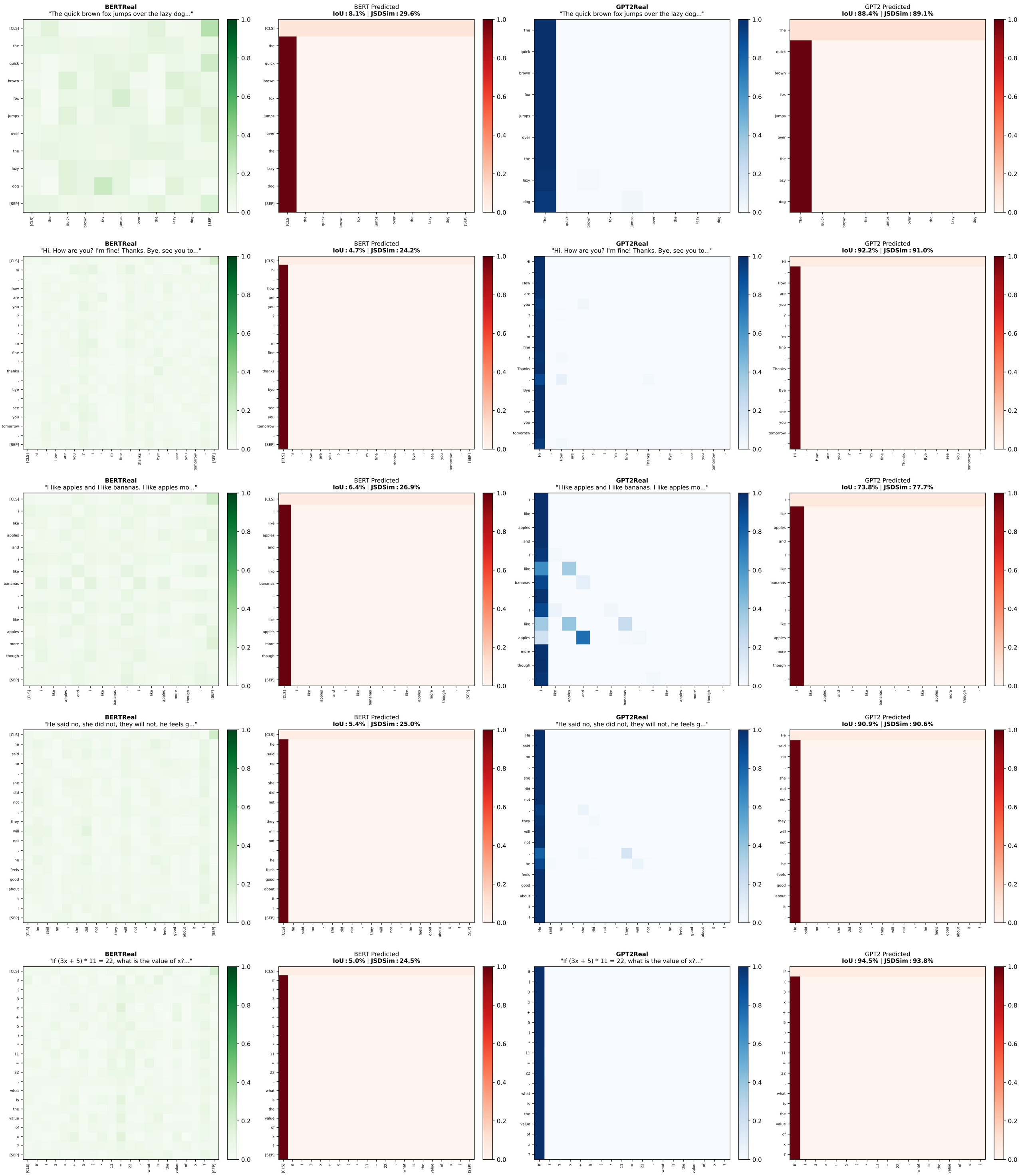
Pattern: sentence_beginning_attention_pattern

```
def sentence_beginning_attention_pattern(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Assign full attention to the first token
    out[0, 0] = 1
    for i in range(1, len_seq-1):
        out[i, 0] = 1
        out[0, i] = 1
    out[-1, 0] = 1

    # Normalize
    out += 1e-4
    out = out / out.sum(axis=1, keepdims=True)
    return "Sentence Beginning Attention Pattern", out
```



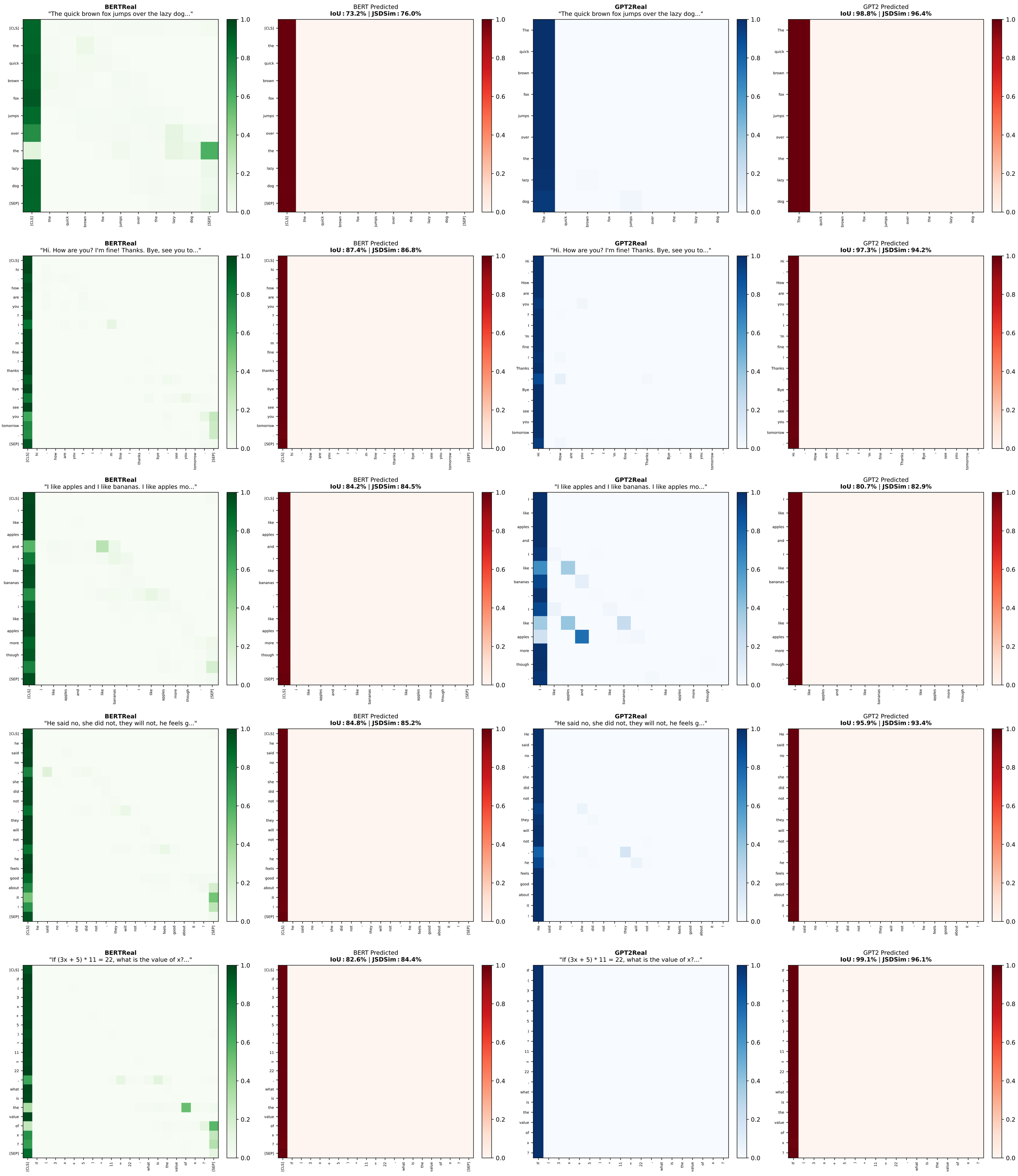
Pattern: sentence_beginning_emphasis

```
def sentence_beginning_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase):
                    toks = tokenizer([sentence], return_tensors='pt')
                             len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
                       out[0, :] = 1  # High attention on the first token
                   out[0, 0] = 1  # Self-focus of the start token to itself
               out[1:, 0] = 1  # Every other token attends to the first token
            out = out / out.sum(axis=1, keepdims=True)  # Normalize attention
                     return 'Sentence-Beginning Token Emphasis', out
```



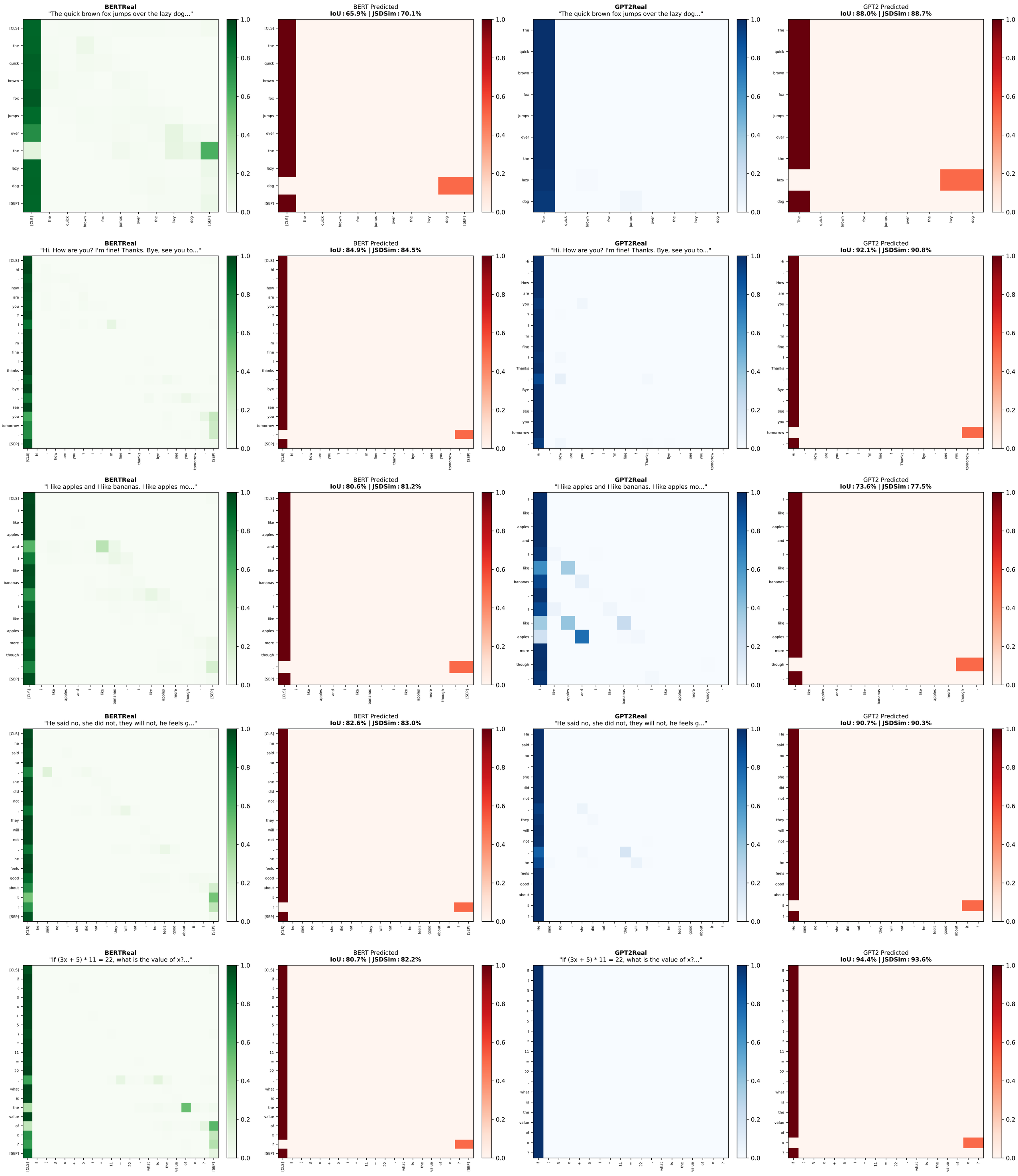
Pattern: sentence_beginning_salience

```
def sentence_beginning_salience(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                    toks = tokenizer([sentence], return_tensors="pt")
                                            len_seq = len(toks.input_ids[0])
                                           out = np.zeros((len_seq, len_seq))

                # Generally gives higher attention/intensity to the first few tokens in the sentence.
                                             for i in range(1, len_seq - 1):
                                               distance_to_start = i / len_seq
                                         salience = max(0, 1 - distance_to_start)
                                                  out[i, 0] = salience

                                    out[0, 0] = 1  # CLS token retains self-attention
                                   out[-1, 0] = 1  # EOS token retains self-attention

        # Normalize attention scores to sum to 1 across each row (excluding last row to mimic padding effects)
                                                          out += 1e-4
                                        out = out / out.sum(axis=1, keepdims=True)

                                         return "Sentence Beginning Salience", out
```



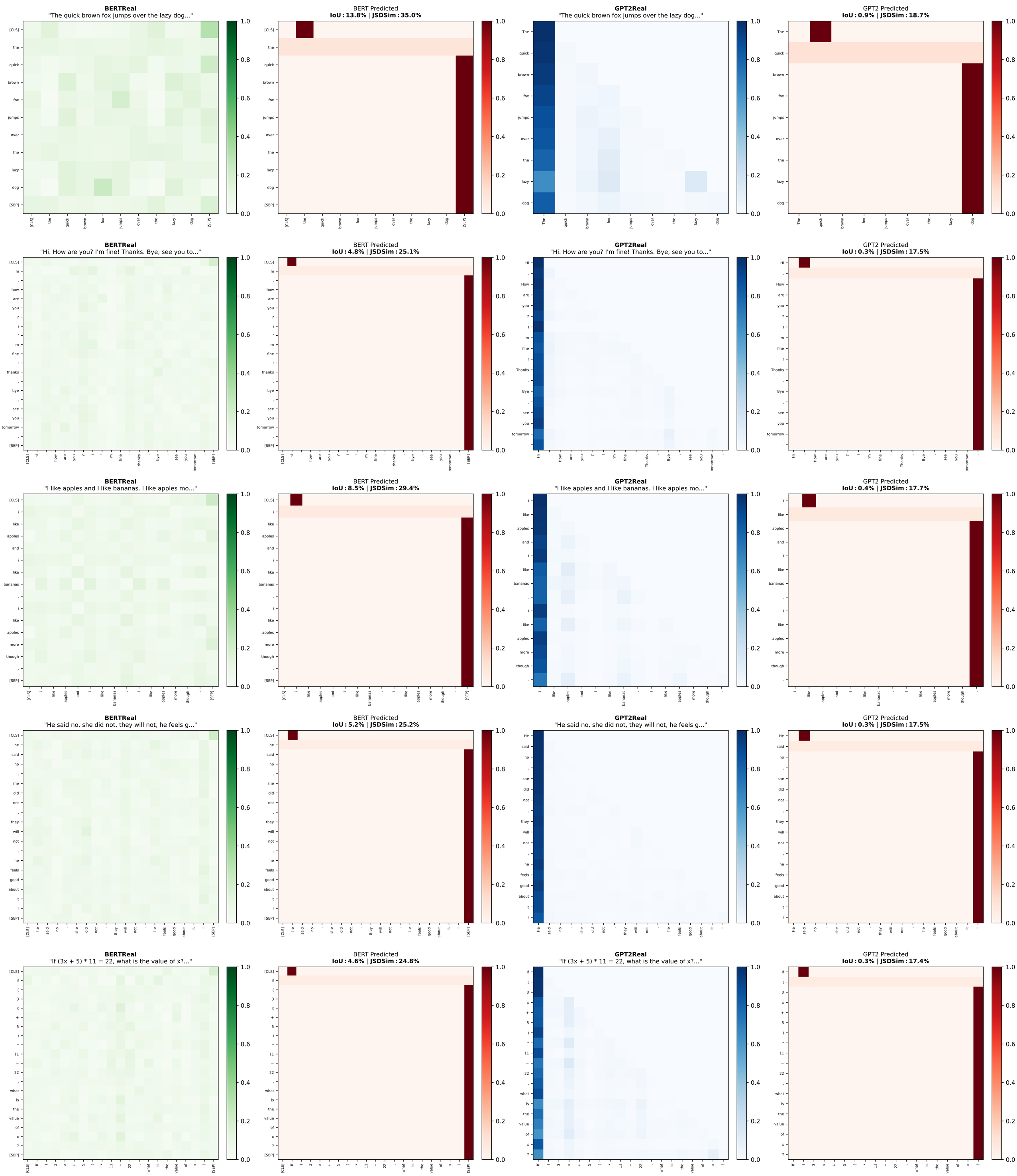
Pattern: sentence_boundary_focus

```
def sentence_boundary_focus(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))

                # Focus attention on sentence boundaries (first and last token)
                       out[0, 0] = 1  # CLS-like self-attention at start
                        out[-1, 0] = 1  # EOS-like self-attention at end

                                for i in range(1, len_seq - 1):
                                        if i == len_seq - 2:
                            # Penultimate token might focus more towards EOF
                                            out[i, i] = 0.5
                                           out[i, -1] = 0.5
                                                 else:
                        # All tokens somewhat uniformly concentrate on the start
                                             out[i, 0] = 1

  # Normalize to make sure each row sums to 1, mimicking probability distributions in attention
                            row_sums = out.sum(axis=1, keepdims=True)
                                       out = out / row_sums

                                  return "Sentence Boundary Focus", out
```



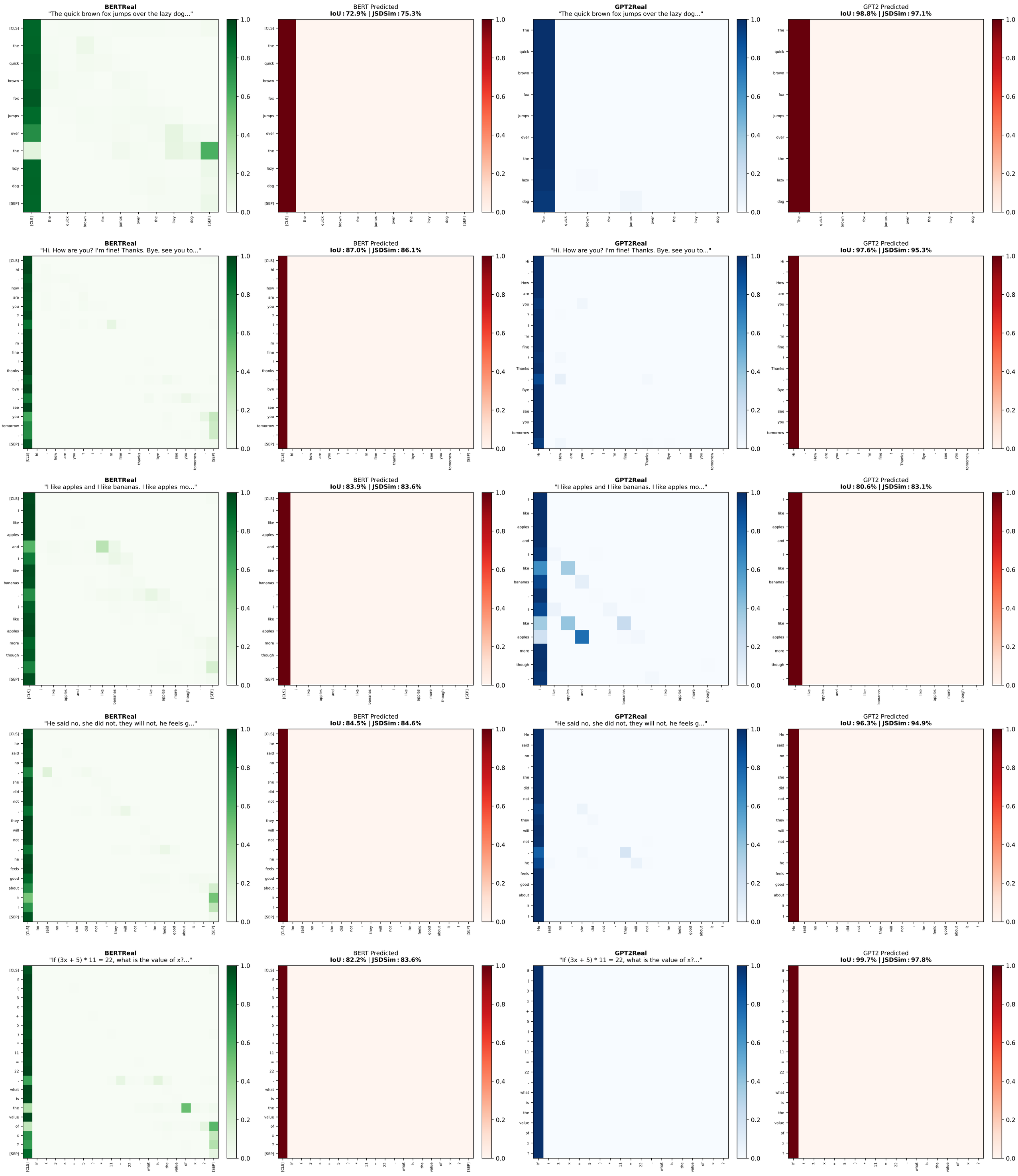
Pattern: sentence_initial_dominance

```
def sentence_initial_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase):
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
        # Assuming CLS token attention to itself and the sentence start token
                                init_token_index = 1
                             out[0, init_token_index] = 1
      out[init_token_index] = 1  # Initial token pays highest attention to all
          # Ensure rest of tokens have at least attention to one end token
                              for row in range(len_seq):
                                  if out[row].sum() == 0:
                                 out[row, -1] = 1.0  # EOS token
             out += 1e-4  # Prevent division by zero during normalization
                out = out / out.sum(axis=1, keepdims=True)  # Normalize
                     return "Sentence Initial Word Dominance", out
```



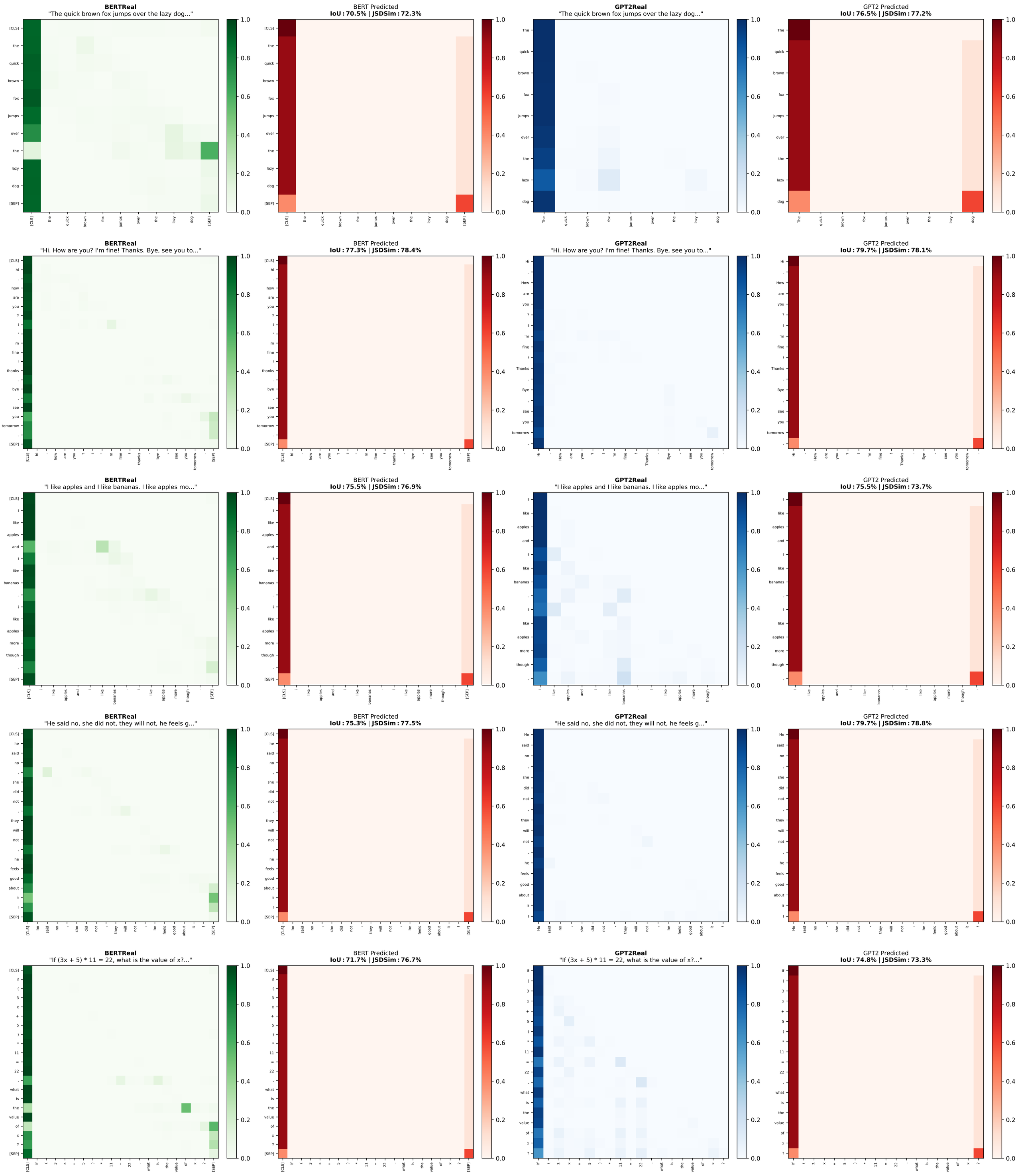
Pattern: sentence_initiation_emphasis

```
def sentence_initiation_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors='pt')
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Assign heavy weights to the first token indicating sentence initiation emphasis
    out[0, 0] = 1
    for i in range(1, len_seq):
        out[i, 0] = 1 / (len_seq - 1)  # Normalize

    # Normalize each row to sum to 1
    row_sums = out.sum(axis=1, keepdims=True)
    np.divide(out, row_sums, where=row_sums != 0, out=out)

    return 'Sentence Initiation Emphasis', out
```



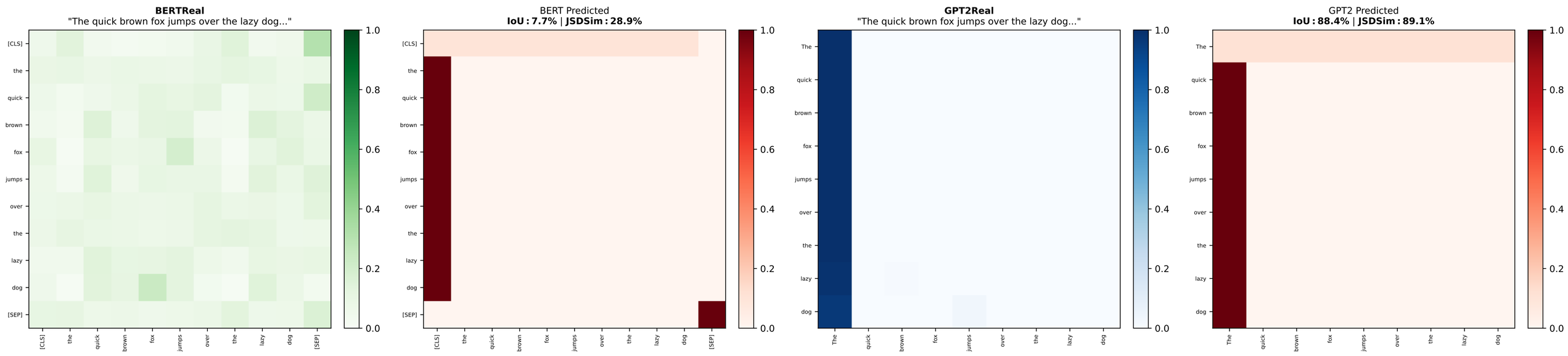
Pattern: sentence_level_attention

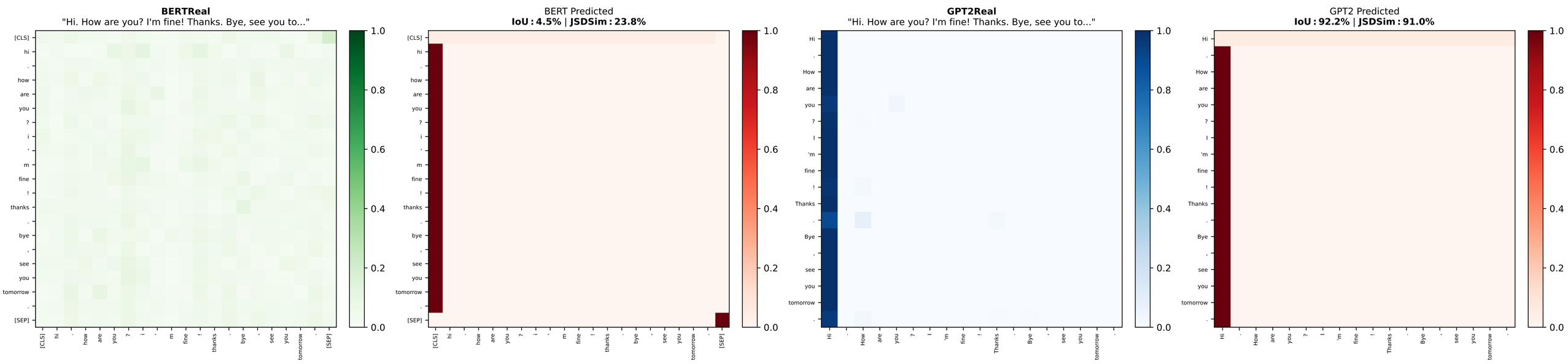
```
def sentence_level_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                        len_seq = len(toks.input_ids[0])
                                       out = np.zeros((len_seq, len_seq))

    # For simplicity, assume the sentence starts with a common structural token and mostly attends there
                            # rather than heavily attending to specific content tokens.
            main_attention_token_idx = 0  # Assume CLS-like attention to the first available token
             secondary_attention_token_idx = len_seq - 1  # Including attention to the last token

                                # Apply sentence-level attention distribution
                                        for i in range(len_seq):
                                                    if i == 0:
                                                     out[i, i] = 1
                                          elif i == main_attention_token_idx:
                                           out[i, main_attention_token_idx] = 0.6
                                         out[i, secondary_attention_token_idx] = 0.4
                                        elif i == secondary_attention_token_idx:
                                           out[i, main_attention_token_idx] = 0.4
                                         out[i, secondary_attention_token_idx] = 0.6
                                                          else:
                                           out[i, main_attention_token_idx] = 0.9
                                         out[i, secondary_attention_token_idx] = 0.1

                                     # Very basic simulation of normalization
                            out += 1e-5  # Avoid division by zero in some implementations
                                        out /= out.sum(axis=1, keepdims=True)

                                 return "Sentence-Level Attention Center", out
```

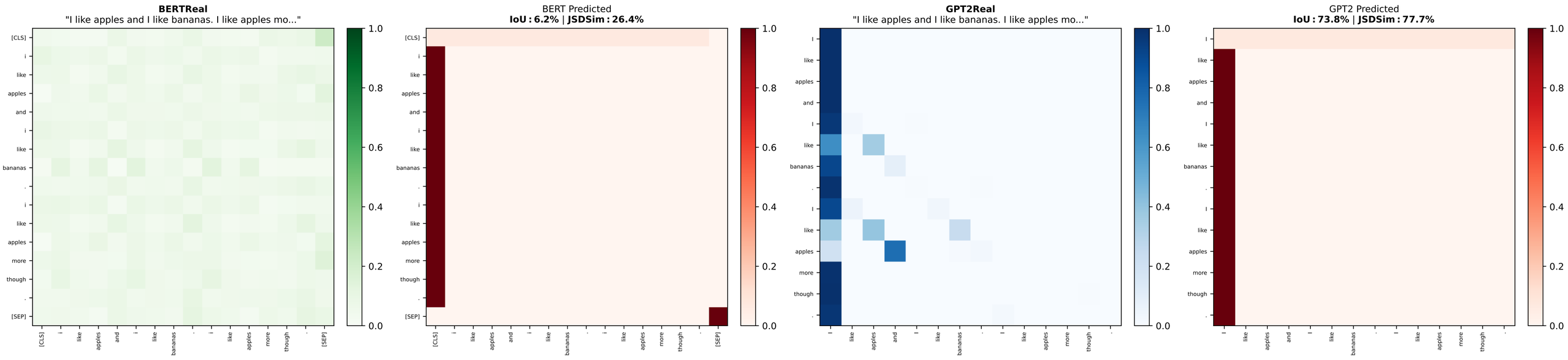


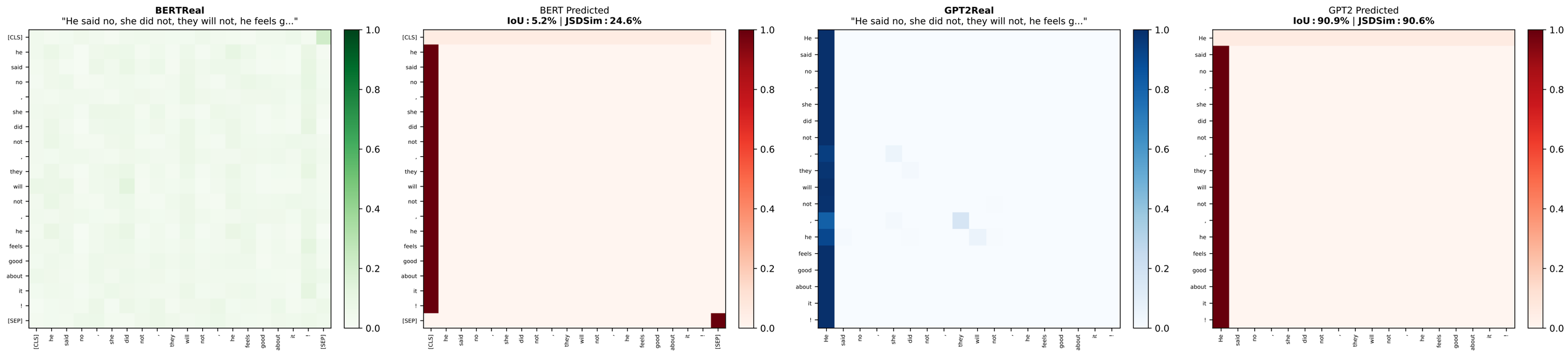
Pattern: sentence_level_initial_token_repetition

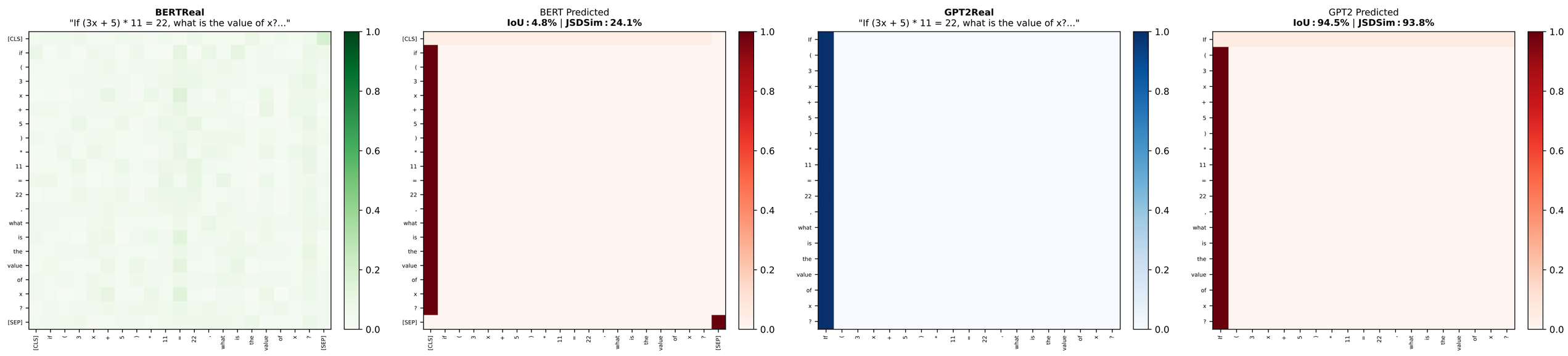
```
def sentence_level_initial_token_repetition(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Tokenize with spaCy for alignment
    doc = nlp(sentence)
    spacy_tokens = [token.text for token in doc]
    spacy_to_hf_id = {i: j for i, j in enumerate(toks.word_ids(0))}

    # Assign attention
    first_token_idx = spacy_to_hf_id[0]
    for i, hf_idx in enumerate(toks.word_ids(0)):
        if hf_idx == first_token_idx:
            out[i, i] = 1
        else:
            out[0, i] = 1
            out[i, 0] = 1

    # Normalize to simulate typical attention behavior
    out = out / out.sum(axis=1, keepdims=True)
    return "Sentence-level Initial Token Repetition Emphasis", out
```





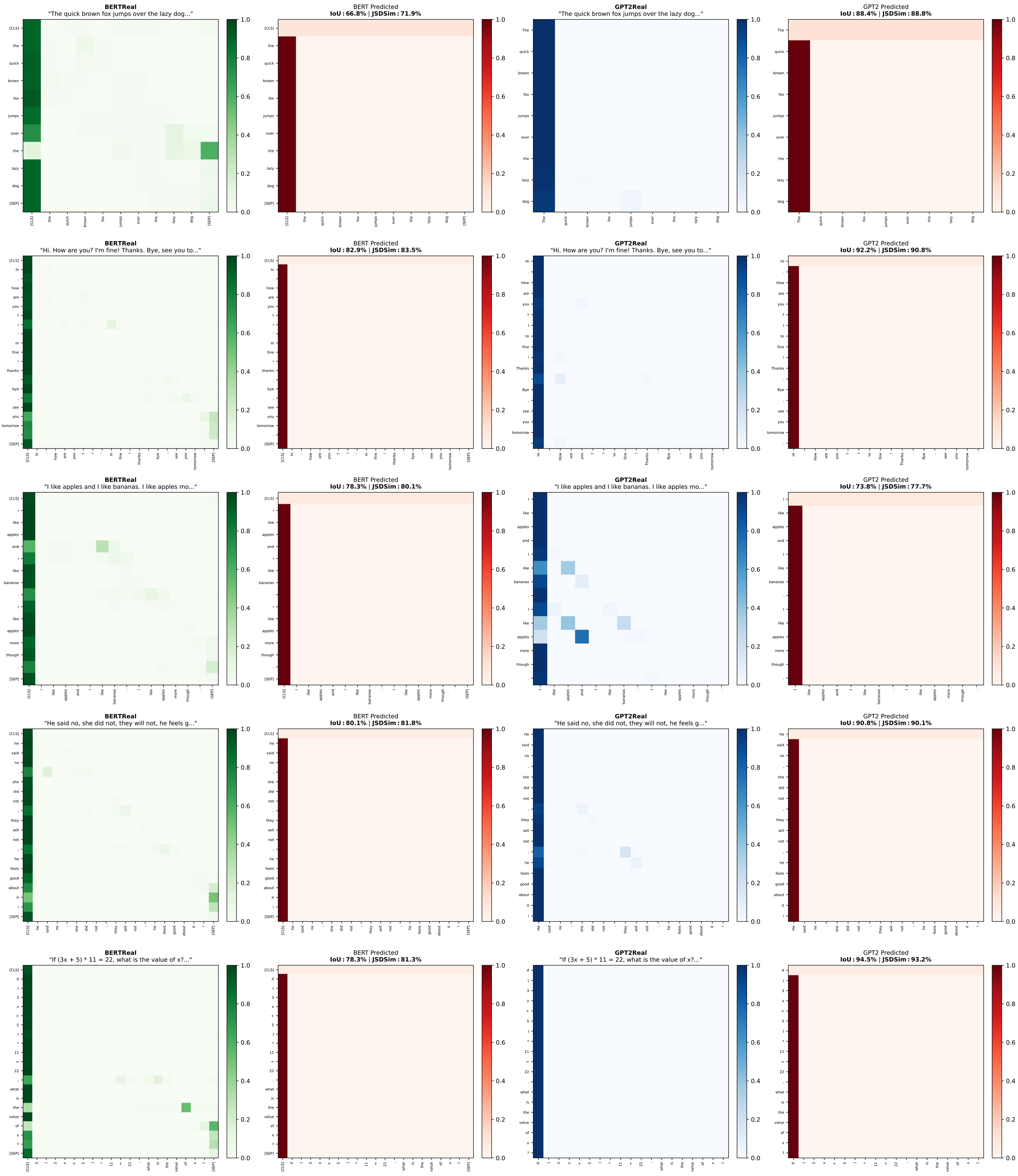






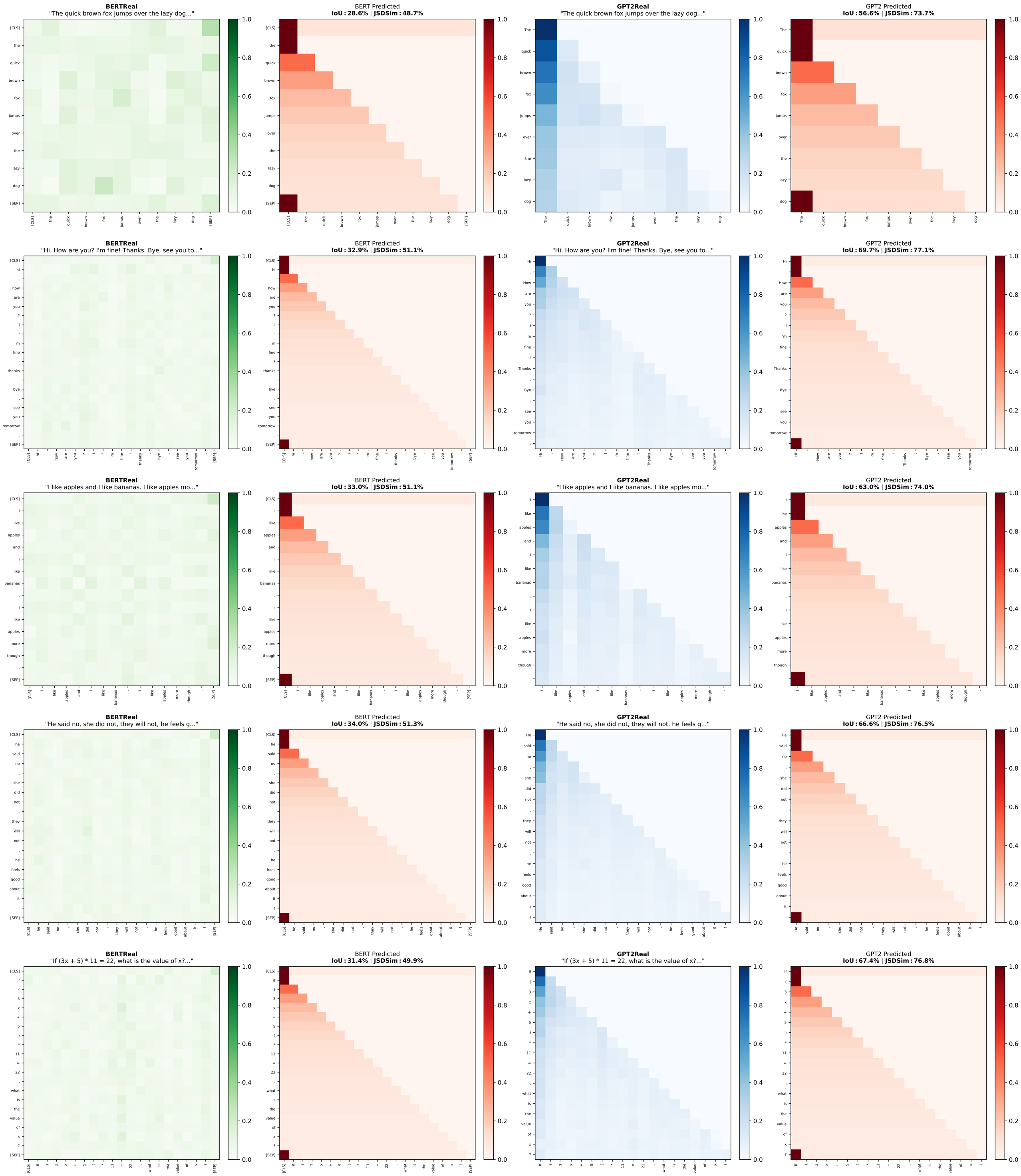
Pattern: sentence_opening_salience

```
def sentence_opening_salience(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                    toks = tokenizer([sentence], return_tensors="pt")
                                            len_seq = len(toks.input_ids[0])
                                           out = np.zeros((len_seq, len_seq))

                                 # Focus primarily on the first token in the sentence
                                out[0, :] = 1  # The first token attends to all tokens
                                 out[:, 0] = 1  # All tokens attend to the first token

                # Ensure each row has some attention to the end token to avoid zero-sum conditions
                                        for row in range(len_seq):
                                            if out[row].sum() == 0:
                                                out[row, -1] = 1.0

                        out += 1e-4  # Add a small value to ensure no rows are zero entirely
                            out = out / out.sum(axis=1, keepdims=True)  # Normalize rows

                                         return "Sentence Opening Salience", out
```



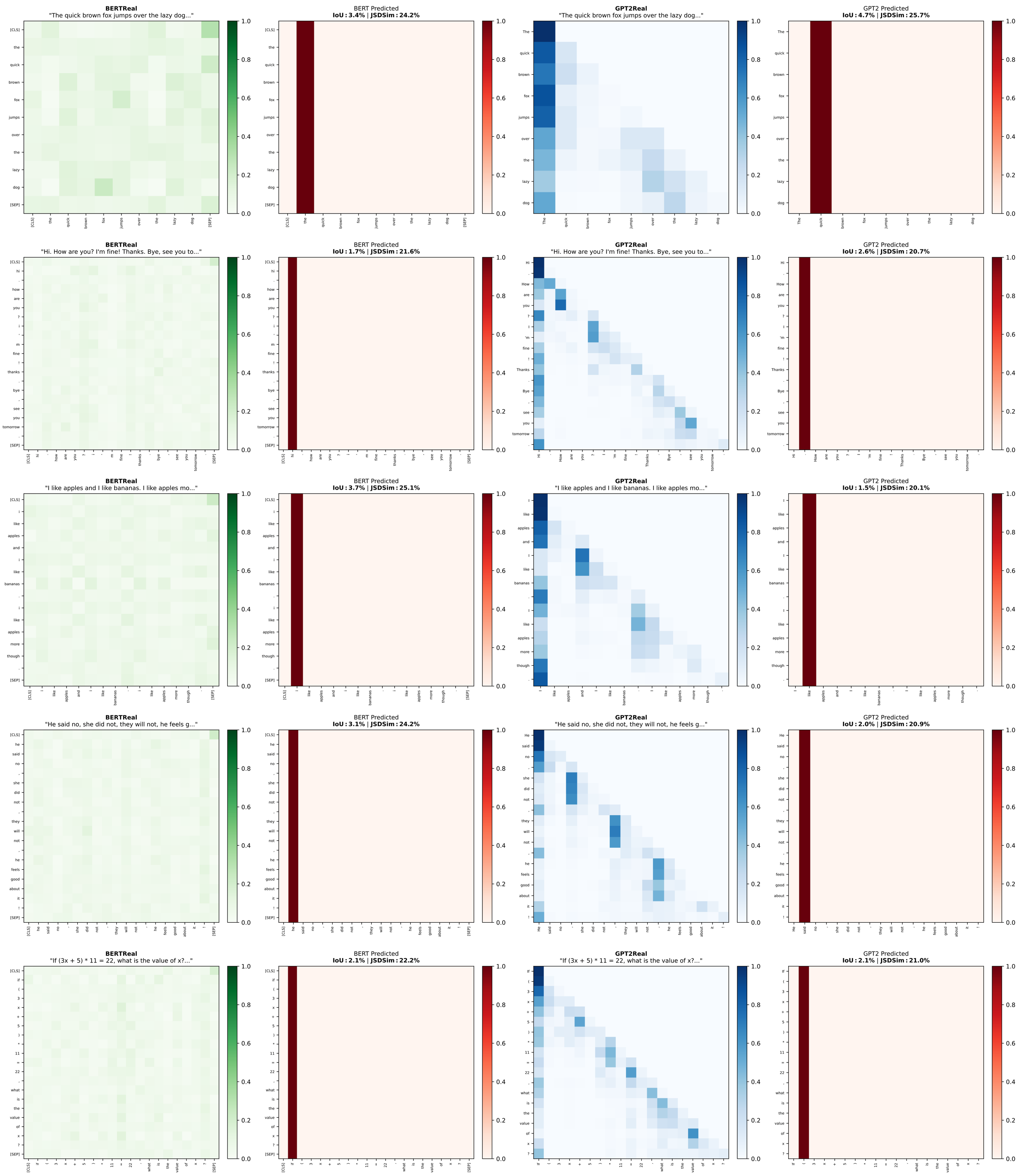
Pattern: sentence_position_preference

```
def sentence_position_preference(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                        toks = tokenizer([sentence], return_tensors="pt")
                                len_seq = len(toks.input_ids[0])
                               out = np.zeros((len_seq, len_seq))

            # Hypothesis is that words earlier in the sentence receive more attention
                              # than words later in the sentence
                                   for i in range(len_seq):
                                inverse_position_weight = len_seq - i
                                        for j in range(len_seq):
          weight_multiplier = 1 if j < i else 0  # Preference for attention to previous tokens
                         out[i, j] = inverse_position_weight * weight_multiplier

                            # Normalize the attention matrix row-wise
                                          out += 1e-4
                              out /= out.sum(axis=1, keepdims=True)

                # Make sure [CLS] and [SEP] tokens receive some base self-attention
                                          out[0, 0] = 1
                                         out[-1, 0] = 1

                             return "Sentence Position Preference", out
```



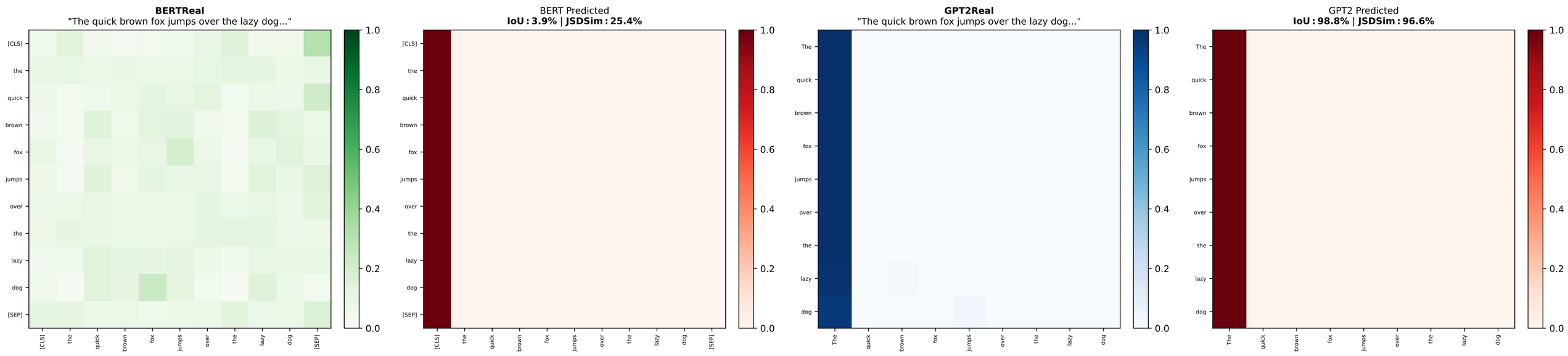
Pattern: sentence_start_attention

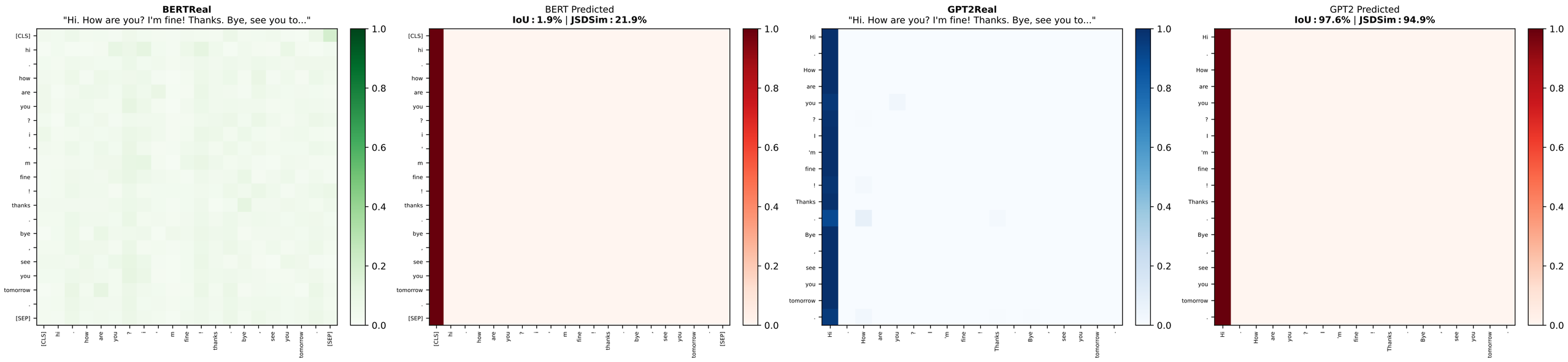
```
def sentence_start_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Set attention pattern
    # Every token attends primarily to the first content token
    for i in range(len_seq):
        out[i, 1] = 1  # Attend to the first non-special token (often a specific content start)

    # Ensure no row is all zeros by giving attention to the [CLS] or starting token
    for row in range(len_seq):
        if out[row].sum() == 0:
            out[row, 0] = 1.0

    out += 1e-4  # Avoid division by zero
    out = out / out.sum(axis=1, keepdims=True)  # Normalize row-wise

    return "Sentence Start Focus", out
```

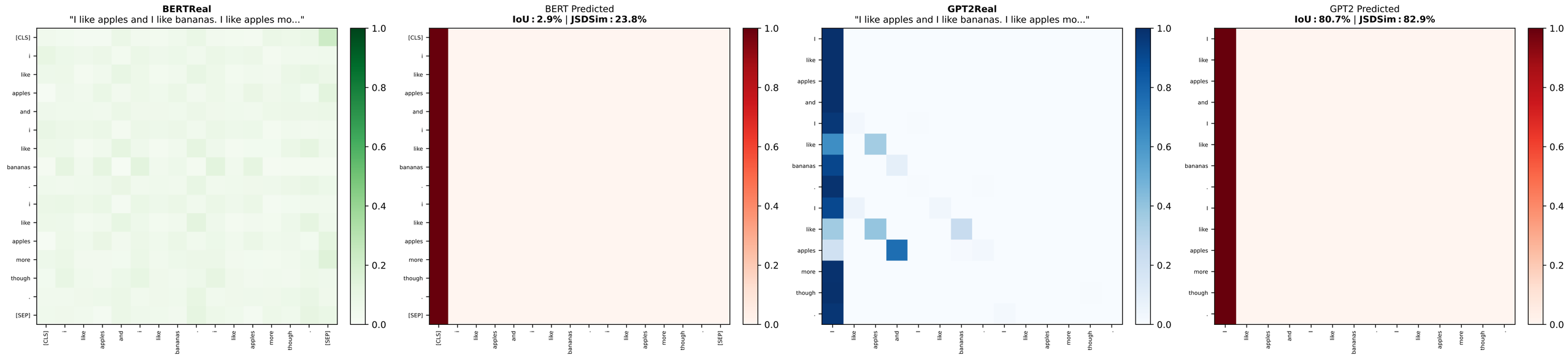


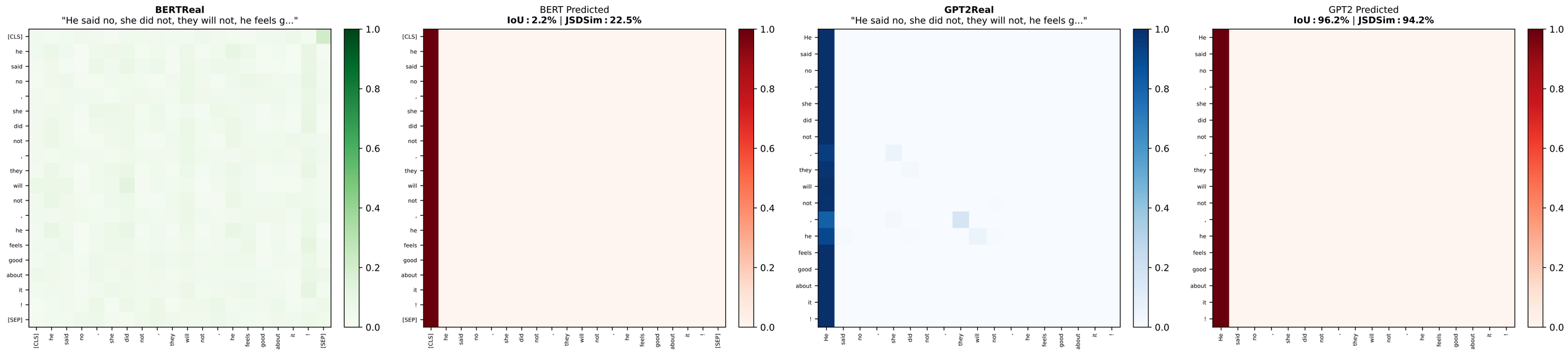
Pattern: sentence_start_dominance

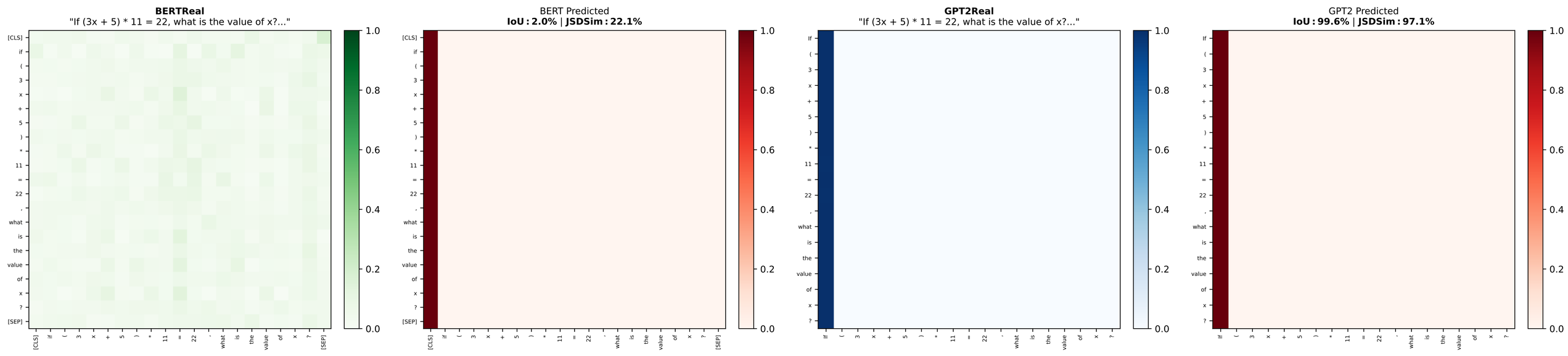
```
def sentence_start_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
# Pattern: All word tokens tend to heavily attend to the first word/token in the sentence
    for i in range(len_seq):
        out[i, 0] = 1
    # Assign cls (out[0, 0] = 1) for emphasis
    out[0, 0] = 1
    # Normalize to simulate attention distribution
    out += 1e-4
    out = out / out.sum(axis=1, keepdims=True)
    return "Sentence Start Dominance", out
```





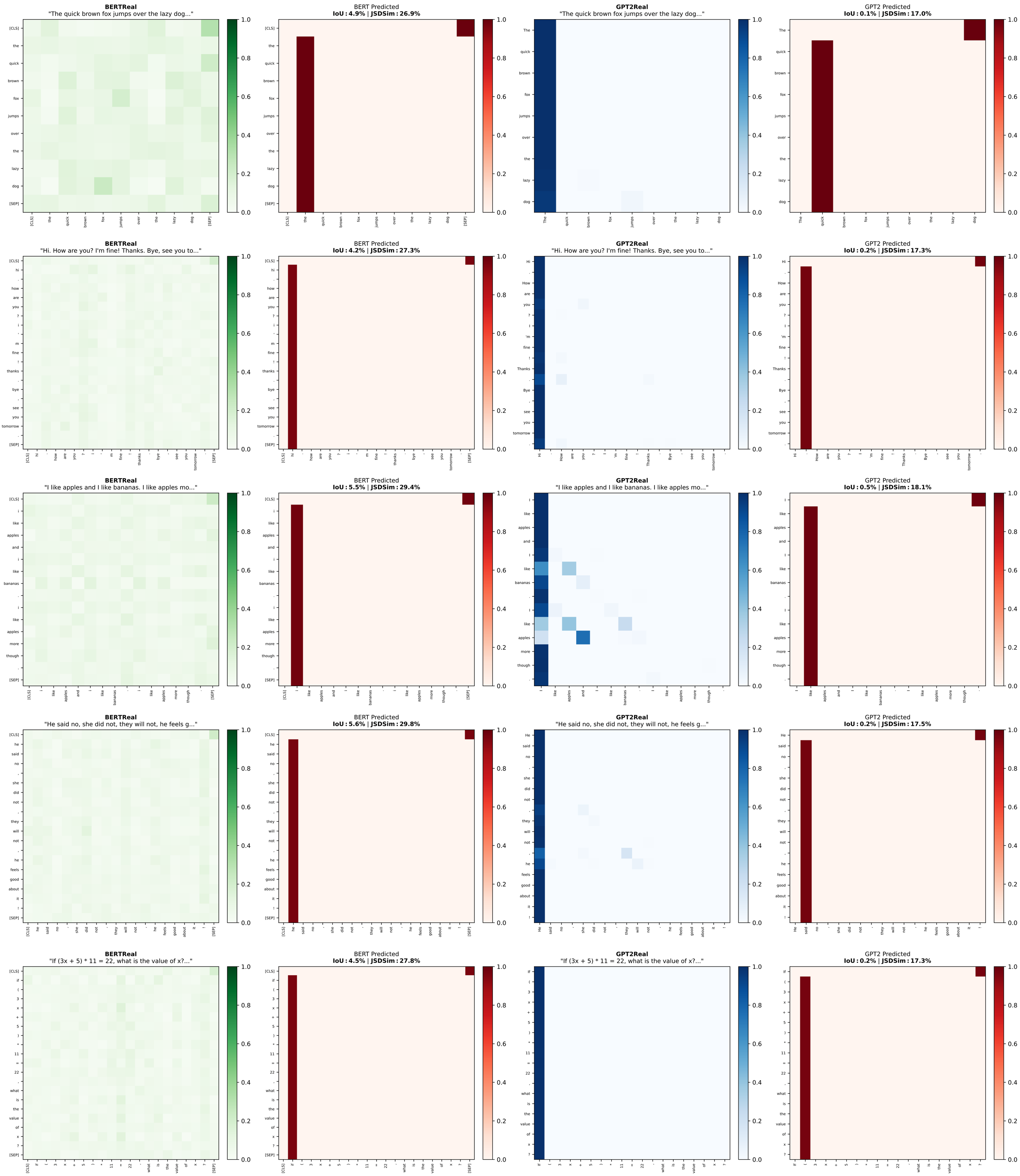






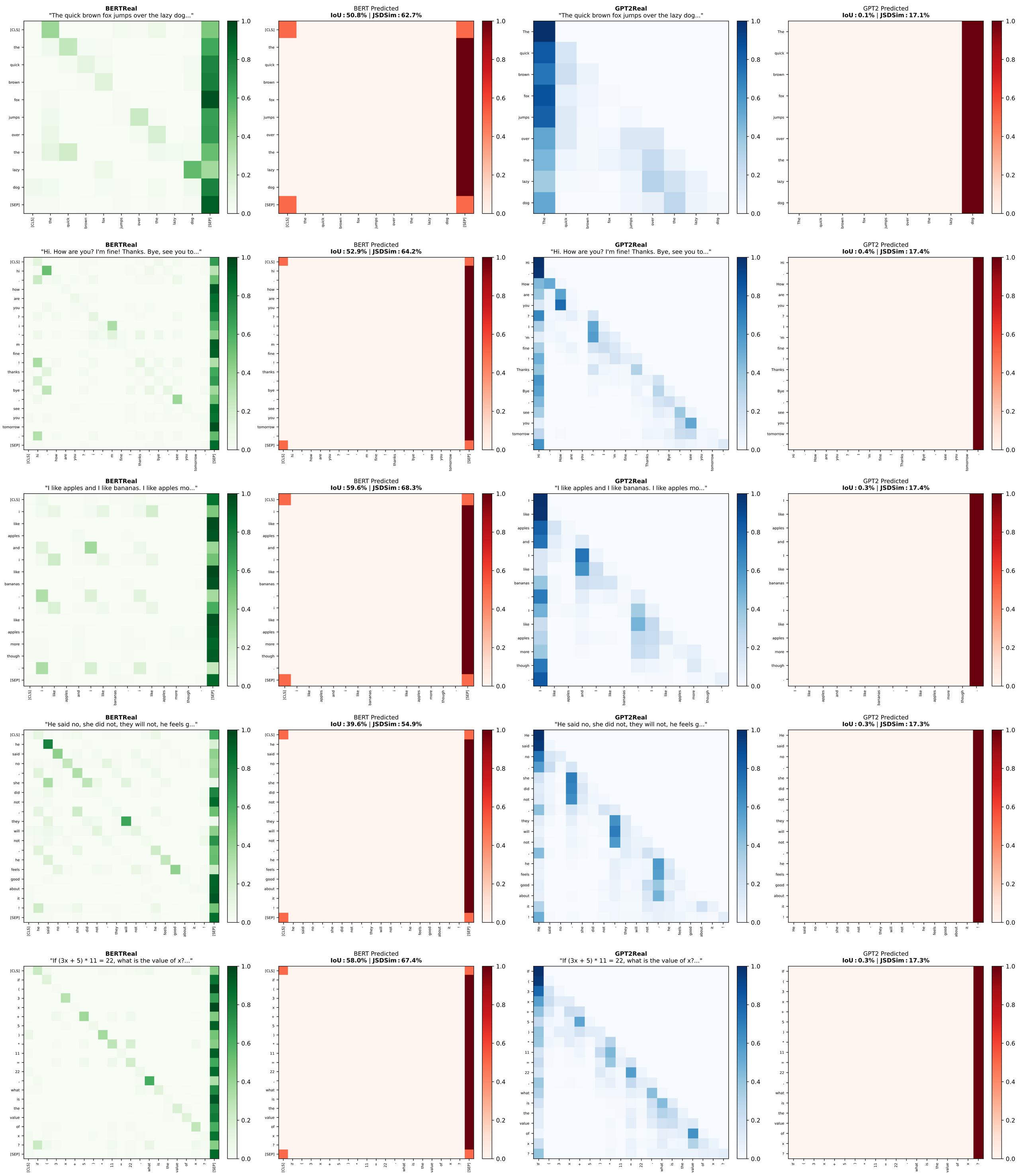
Pattern: sentence_start_emphasis

```
def sentence_start_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                        len_seq = len(toks.input_ids[0])
                                       out = np.zeros((len_seq, len_seq))

                    # Emphasize attention towards the initial tokens of clauses/sentences
                                         for i in range(1, len_seq):
                 out[i, 1] = 1  # Assume the first actual token after CLS gets highest attention

                                  # Ensure attention distribution consistency
                                         for row in range(len_seq):
                                            if out[row].sum() == 0:
                      out[row, -1] = 1.0  # default to padding token if no attention is placed
                                        out += 1e-4  # Avoid division by zero
                         out = out / out.sum(axis=1, keepdims=True)  # Normalize by rows

                                 return "Sentence Start Emphasizing Pattern", out
```



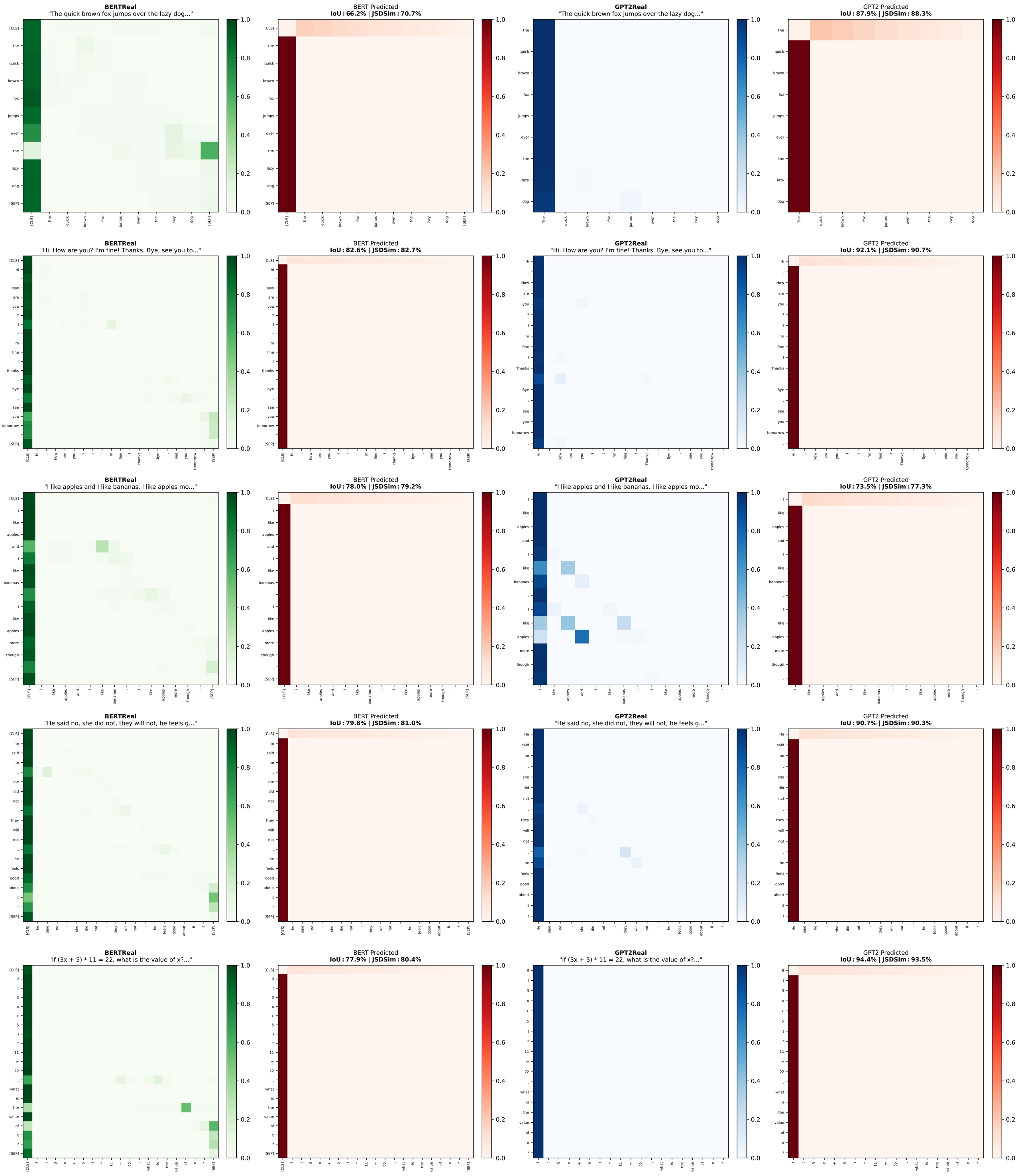
Pattern: special_token_attention

```
def special_token_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    special_tokens = tokenizer.all_special_ids
    for i in range(len_seq):
        if toks.input_ids[0][i] in special_tokens:
            for sp_tok in special_tokens:
                out[i, toks.input_ids[0] == sp_tok] = 1
            else:
                out[i, -1] = 1
    out = out / out.sum(axis=1, keepdims=True)
    return "Special Token Pattern", out
```



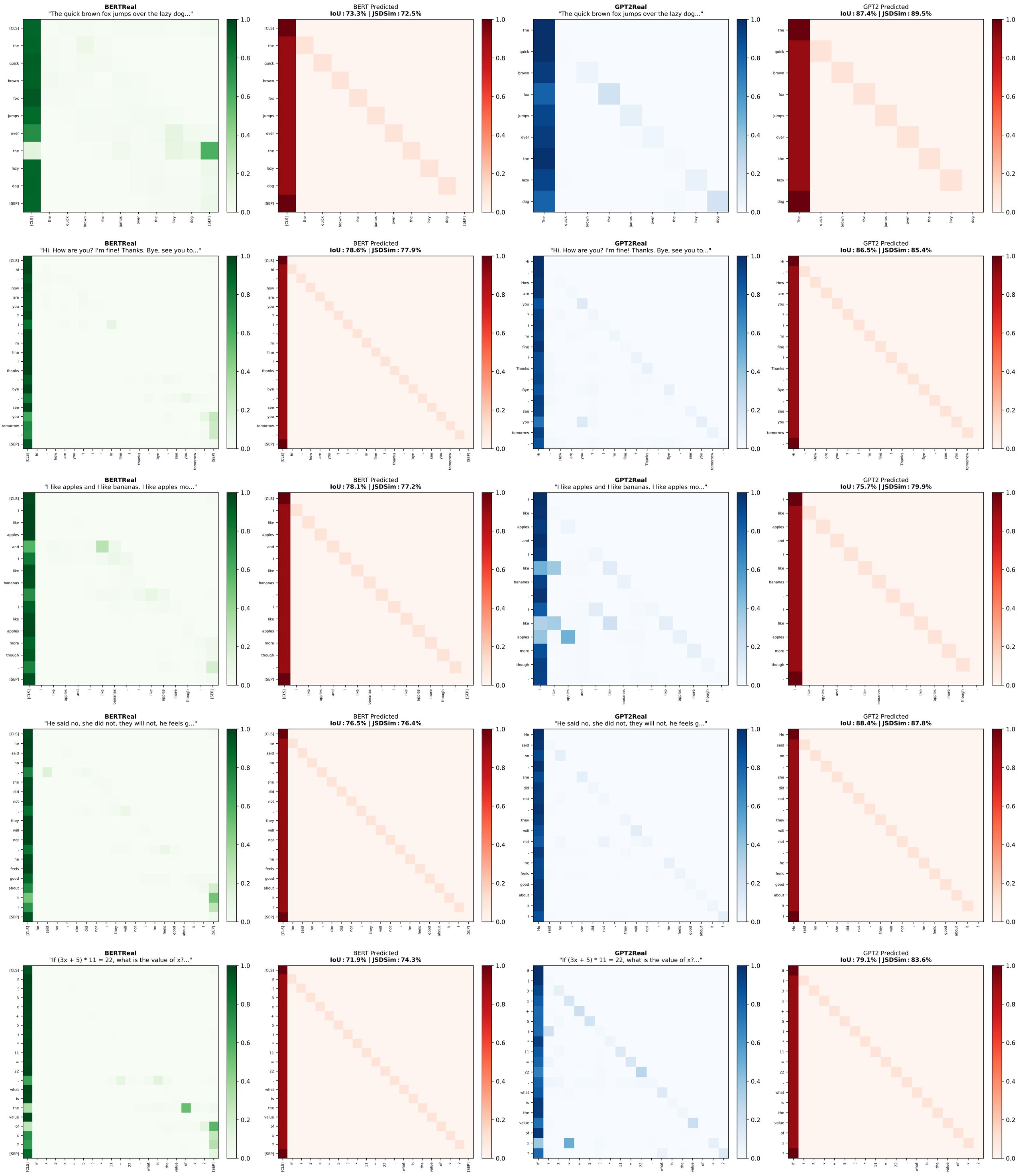
Pattern: token_emphasis_subsequent_dominance

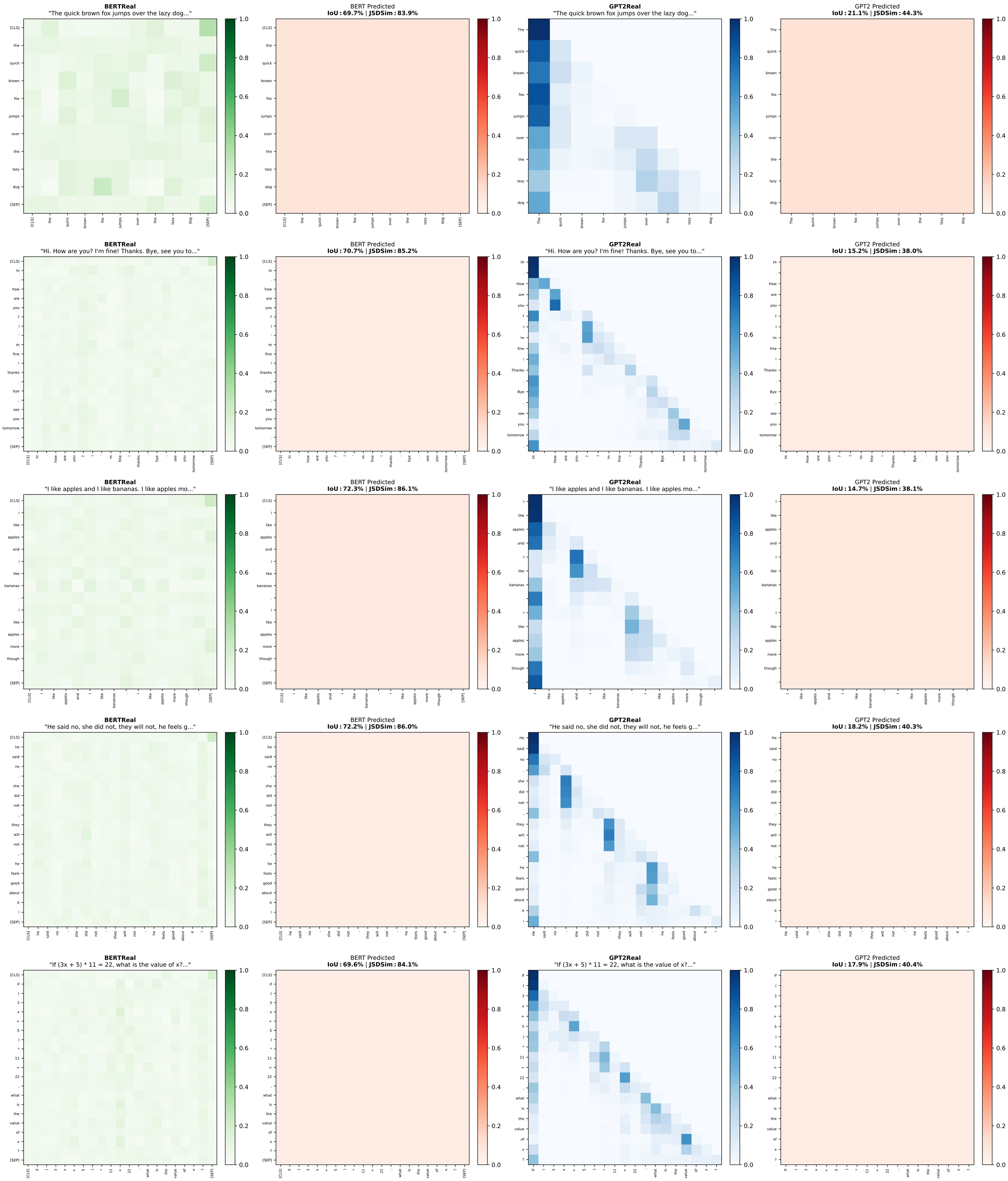
```
def token_emphasis_subsequent_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    first_token_index = 0  # The starting [CLS] token
    end_token_index = len_seq - 1  # The closing [SEP] or [EOS] token

    # Assign high attention weight to the initial token [CLS] across all sentence tokens
    out[:, first_token_index] = 1
    out[first_token_index, first_token_index] = 1

    # Assign subsequent high weights from initial token to others, but diminish over sequence length
    for i in range(1, len_seq):
        out[i, first_token_index] = len_seq - i
        out[first_token_index, i] = len_seq - i

    # Normalize to ensure each row sums to 1, achieving a probabilistic attention matrix
    out /= out.sum(axis=1, keepdims=True)

    return "Initial Token Emphasis with Subsequent Token Dominance", out
```



Pattern: token_reinforcement

```
def token_reinforcement(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                        len_seq = len(toks.input_ids[0])
                                       out = np.zeros((len_seq, len_seq))

  # Identify the main token to reinforce attention (this model shows inclination at the sentence start)
                          # Anchors special tokens like the sentence start and end
                         out[0, 0] = 1  # reinforce attention to the opening token

      # Reinforce each non-special token to start attention back to the first word of the sentence
                                        for i in range(1, len_seq - 1):
                                                 out[i, 0] = 0.9

                                        # Add self-attention weightage
                                        for i in range(1, len_seq - 1):
                                                 out[i, i] = 0.1

                     out[len_seq - 1, 0] = 1  # End token attention back to start
               out += np.eye(len_seq) * 1e-5  # Small added value to ensure no zeros (optional)

                          # Normalize rows for attention probabilistic pattern
                               row_sums = out.sum(axis=1, keepdims=True)
                                            out /= row_sums

                 return "Initial Token Reinforcement with Sentence Anchoring Pattern", out
```



Pattern: uniform_attention

```
def uniform_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.ones((len_seq, len_seq)) / len_seq
    return "Uniform Pattern", out
```